OTA-Shield: A Hardware-Measured Reference Architecture for In-Network OTA Rollout Admission and Bounded Firmware-Attack Detection in Battery Energy Storage Systems

ARTICLE INFO

Keywords:
Battery energy storage
OTA firmware
programmable data plane
Intel Tofino
intrusion detection
critical infrastructure

ABSTRACT

Battery energy storage systems (BESS) depend on over-the-air (OTA) firmware updates pushed across a fleet of battery management systems (BMS). A coordinated OTA campaign is visible on the network transit path, yet endpoint signature verification cannot see it and current BESS defenses do not use this signal. We present OTA-Shield, a hardware-measured reference architecture that admits authorized firmware rollouts and operator rollbacks and detects a bounded family of OTA firmware attacks on a cleartext reference MQTT channel. An Intel Tofino 1 pipeline parses the OTA header and tracks per-BMS state in the data plane; a control-plane Rollout-Authorization Table (RAT) and a two-stage arbiter check observed events against the operator's rollout schedule, admitting scheduled rollouts and operator rollbacks instead of dropping them.

On a 50-unit emulated BMS testbed the detector classifies the designed attack family of replay and unauthorized-source campaigns at $F_1 = 0.996$, with no false alarm on the benign traffic in that run (0 of 600 benign events in E8; Clopper–Pearson 95% upper bound 0.50% on the false-positive rate). The median end-to-end decision latency is 34.0 ms, well inside the minutes-to-hours window over which an OTA campaign reaches a fleet. Where a stateful signature-IDS baseline flags benign rollouts as attacks, OTA-Shield does not, because the fleet-fanout rule counts only unauthorized-source fanout; a CPU port of the same rule-and-arbiter logic matches detection quality exactly, leaving only throughput and resource cost as the difference; switch line-rate is not measured.

Detection requires a cleartext MQTT segment after TLS or VPN termination, evaluated as a single reference channel profile. Because the fleet-fanout rule charges only on unauthorized sources, an attacker who holds a RAT-authorized source and version and stays inside the rollout envelope is outside its scope; there the arbiter admits authorized operations and rollbacks. The headline F_1 applies to this designed attack family; against an adaptive adversary who reshapes cadence and volume, coverage falls to 1.7% of attack events.

1. Introduction

Battery energy storage is becoming a critical grid asset. Compound annual growth in installed capacity currently runs at 30 to 45% in major markets [1]. Every installation depends on over-the-air (OTA) firmware updates pushed to its battery management system (BMS) fleet, typically over MQTT or a proprietary equivalent. When that channel is compromised, an attacker can flash malicious firmware to dozens or hundreds of BMS units at once. Fleet-wide BMS misbehavior has documented grid-impact pathways: modelled frequency-response loss [2], thermal runaway as documented in the McMicken 2019 and Moss Landing 2024 incidents (neither attributed to OTA compromise), and supply withdrawal during peak demand [3]. We do not claim any of these events was caused by OTA compromise. Quantitative grid-impact study is out of scope; the documented attack surface and the documented impact pathways are independently sufficient motivation.

Current practice secures the endpoint, not the transit path. SUIT (RFC 9019) [4] and Uptane [5] prevent an individual device from installing unsigned firmware, but neither constrains the fleet fanout rate of validly signed images. Uptane's release-counter and director-repository design constrain which image a target may install, not the fanout pattern used to reach the fleet, so in-transit detection is complementary rather than a replacement. IEEE 2030.5-2018 mandates mutual TLS but does not require online OCSP/CRL; field deployments often omit revocation, and Sandia/SunSpec implementation guidance leaves revocation handling to the deploying aggregator [1, 6]. A stolen certificate retains operational validity until that aggregator's policy revokes it, so revocation latency is the relevant attack window. CPU-based signature IDSes such as Suricata and Snort 3 were built around stateless or short-horizon rules, and the long-horizon per-BMS state needed to correlate an OTA campaign is awkward to express in their rule languages [7, 8].

This leaves a specific gap. A coordinated OTA campaign is visible on the network transit path, where the fleet-fanout pattern is exposed, yet it is invisible to endpoint signature verification, which sees each device in isolation. Existing BESS security research either models attack surfaces without proposing a detector [2, 9], or places detection on the device through side-channel or telemetry ML [10]. None of these approaches observe

the network transit path, which is where coordinated campaigns become visible.

Programmable switches provide a per-packet vantage point for that observation. An Intel Tofino 1 pipeline can parse MQTT, extract an OTA header, maintain per-flow counters, and fire detection rules inside the ASIC forwarding path [11]. We use the Tofino 1 for its programmability, not as a throughput claim: switch line-rate is not measured in this paper (see §7). Earlier work applied these primitives to volumetric DDoS [12, 13] and generic per-flow ML [14, 15], but we are not aware of any prior work targeting application-layer OTA firmware attacks on BESS infrastructure.

This paper describes OTA-Shield. The contribution is an OTA anomaly detector for BESS fleets that runs its rule checks in a Tofino 1 programmable data plane and its admission logic in a small control-plane arbiter: four rules compile into the data plane (R2 unauthorized source, R4 oversize, R5 unauthorized-source fanout, R6 per-BMS version monotonicity), and the arbiter consults a Rollout-Authorization Table so that scheduled rollouts and operator rollbacks are admitted instead of dropped. Only the terminal R2 drop is resolved entirely in-switch at line rate; R4 and R6 (and R1 where it applies) raise a HOLD that the controller resolves in the loop, so we make no line-rate throughput claim. We use the Tofino 1 for early, fail-closed filtering and to compile the rule set into a fixed resource budget. The terminal attack rules (R2, R4, R6) need no control-plane help to *detect*; the arbiter handles the harder case, separating an authorized rollback or re-push from a rollback attack or a replay, which we evaluate directly in E19 and E12. We present OTA-Shield as a hardware-measured reference design for in-network OTA detection on Tofino 1, evaluated end-to-end on a 50-unit emulated BMS testbed with a single reference MQTT profile. Two scope boundaries apply throughout. First, OTA-Shield reads cleartext OTA fields, so it must sit on a plaintext segment after TLS or VPN termination. Second, the deployed fleet-fanout rule (R5) counts only unauthorized-source fanout, so an attacker who holds a RAT-authorized source identity and firmware version and operates inside the authorized rollout envelope is outside the scope of R5-based detection. Coordinated fanout of validly signed firmware from a stolen but authorized key is thus the motivating case, which we report as out of scope rather than as a result: on the deployed build the delivered detection contribution is R6 version-monotonicity (catching a signed-but-older image that endpoint verification accepts) together with the arbiter's false-positive-free admission of authorized rollouts and rollbacks. The evaluation isolates the arbiter's contribution, reports each rule's blind zone under mimicry, compares against Suricata and Zeek on identical traffic, and measures parser portability bounds, scoping all claims to the reference profile measured here. Our contributions are:

- A **two-stage RAT arbiter** (Gate A, Gate B) that consults a Rollout-Authorization Table to admit authorized rollouts and rollbacks without false positives. Its necessity is established on the operations it admits: authorized rollback admission (E19) and benign staged rollouts (E12). On the deployed gated build, disabling the arbiter leaves attack precision unchanged (E4), because the data-plane gate already keeps authorized rollouts out of the fleet-fanout rule (R5); the arbiter's role is admission, not an attack-precision delta.
- An **OTA-aware P4 pipeline** that extracts a 20-byte OTA header from MQTT PUBLISH on Tofino 1 and supplies three data-plane rules to the policy engine: R2 (unauthorized source), R4 (oversize), R5 (fleet fanout, telemetry-gated behind the terminal R2 drop on the deployed build), plus a per-BMS version-monotonicity register (R6). R1 (rapid replay) is cadence-gated: under realistic benign BMS cadences it does not false-fire, and in fast-cadence deployments it is compiled out below $\tau_{R1} = 14\,400$ s; where it is enabled it supplies the rapid-replay half of the E8 headline F_1 [16].
- An **R4 threshold derived from a 90-minute baseline trace** at measured percentiles (P99+$k\sigma$ [17, 18]). R5 (= 4) and R1 ($\tau_{R1} = 14\,400$ s) are compile-time operator-policy constants informed by the baseline trace but not derived from its tail statistics.
- A **hardware-measured evaluation on a 50-unit emulated BMS testbed** covering ablation, adversarial sweeps, benign-rollout stress, held-out mimicry, override-table capacity, parser portability, and a Suricata/Zeek baseline.
- A **deployment-boundary characterization of programmable in-network OTA security**. Beyond the designed-family success case, it measures the limits set by adaptive mimicry (1.7% coverage, E21), brokered-MQTT source collapse (E20), TCP segmentation (E23), parser variation (4/8 variants, E18), and digest-channel and override-table capacity (E11, E13). We report this as a measured boundary rather than a robustness claim: it states where the architecture holds and where an extension is required.

Scope and transferability. The OTA channel used throughout is a reference channel (MQTT over TCP/1883 with a 20-byte application-level OTA header), not a specific vendor implementation. What transfers to other profiles is the construction: a PHV-aligned OTA-header parse, a fleet-cardinality counter (R5), a per-BMS version high-water register (R6), and a two-stage RAT arbiter (Gate A, Gate B).

Each of these is implementable on any $P4_{16}$ target with equivalent register primitives. What does not transfer verbatim are the numerical thresholds and the MQTT framing: R1's interval is tied to per-BMS update cadence, and the parser is bound to the reference framing measured in E18. The throughput measurement is scapy-limited to $\approx$2,000 pps on our generator and is reported as a controller digest-channel sustainable-rate lower bound, not a switch line-rate proof; §7 discusses the further limits this entails.

2. Background and Threat Model

2.1. BESS OTA architecture

A grid-connected BESS installation comprises 10–500 battery modules, each governed by a BMS controller that regulates charging, discharging, and thermal protection [9]. An OTA server operated by the battery vendor or system integrator pushes firmware images to the BMS fleet over an IP network. Commercial vendors vary in how they implement this channel: some use proprietary HTTPS endpoints, some use MQTT, and some use vendor-specific protocols layered on IEEE 2030.5 [19, 20].

For the purposes of this work we adopt a reference OTA channel rather than a specific vendor implementation. The channel uses MQTT on TCP port 1883 for firmware distribution, with each PUBLISH carrying a 20-byte application-level OTA header (magic signature, firmware version, advertised size, and a hash hint). This header format is not proprietary to any one deployment; it captures the minimum semantic information the network needs to recognize an OTA flow. We treat the reference channel as a plausible synthetic deployment target, not as a universal model of real BESS OTA practice. The techniques we develop (protocol-aware extraction in the data plane, fleet-cardinality tracking, and policy-aware arbitration) transfer to any channel that exposes equivalent semantic fields; ports and header layouts are configuration.

2.2. Regulatory gap

NERC CIP standards govern cybersecurity for bulk electric system assets, yet smaller distributed energy resources (DERs), including most BESS installations under 75 MVA, fall outside CIP's mandatory scope [21]. The NERC RSTC white paper on DER cybersecurity acknowledges this gap and recommends voluntary controls, but compliance remains sparse [1]. IEEE 2030.5-2018 mandates TLS mutual authentication for DER aggregator communications. The standard does not specify a revocation profile that operators can use to invalidate a stolen device certificate on the OTA path inside operational SLA windows, and Sandia/SunSpec implementation guidance leaves revocation handling to the deploying aggregator [6]; absent such a profile, a compromised certificate persists across the deployment until manual rotation [1]. NFPA 855 references UL 2900-1 for network-connectable product security, but its primary concern is fire protection, not network-transit defense [22]. IEC 62443 provides a defense-in-depth framework for industrial automation, yet prescribes no specific mechanism for detecting coordinated firmware distribution campaigns [23].

2.3. Threat model and attack taxonomy

The adversary has access to the OTA distribution channel and a valid or stolen signing key, consistent with documented supply-chain compromises [2, 3, 24, 25]. Endpoints are assumed to verify signatures; Table 1 states the full capability, vantage, and scope boundaries.

Table 1
Threat model summary.

Assumption	Detail
Attacker capability	On-path access to the OTA channel (server compromise, stolen TLS cert, or aggregation-point injection) with a valid or stolen signing key.
Attacker goal	Distribute malicious firmware to a BMS fleet quickly enough to affect grid operation before operator intervention.
Defender vantage	Programmable switch on the OTA distribution path (in transit, not on endpoint).
Out of scope	Signing-key cryptanalysis; endpoint exploitation independent of OTA; full TCP reassembly; encrypted-MQTT variants that hide the OTA header from on-path inspection (IEEE 2030.5 mandates TLS; OTA-Shield requires a cleartext segment after TLS/VPN termination); adversaries with valid RAT-whitelisted source IPs and firmware versions operating within authorized rollout parameters (R5 Bloom inserts are gated on unauthorized-source flag, so compliant behavior from a stolen-credential attacker passes R5 undetected).

Within this model five data-plane-observable attack patterns motivate our detection rules, with a separate control-plane concern (OTA-header protocol anomalies) handled by the Python controller rather than in the pipeline:

A1 Fleet fanout Firmware pushed to an anomalously large number of distinct BMS units within a short window, exceeding any authorized rollout pace. Endpoint verification cannot see this pattern. Detected by rule R5 only when the fanout originates from an unauthorized source; authorized-source fanout inside the RAT envelope is out of scope (§6.4).

A2 Unauthorized source Firmware originates from an IP not listed in the operator's Rollout-Authorization Table. Detected by rule R2.

A3 Oversized payload A single MQTT session carries more bytes than the maximum legitimate

firmware image size. Detected by rule R4. R4 uses a P4 range-match against a compile-time size table and therefore exposes a bounded dead zone between the legitimate maximum and the next table boundary (measured at 64 KiB, covering 2.0–2.0625 MiB; §6.5) through which an attacker can slip an oversize but in-dead-zone payload.

A4 Rapid replay The same BMS receives a second firmware push within a window shorter than any legitimate maintenance cycle (typically 4–24 hours). Intended to be detected by rule R1, which is effective only when per-BMS update cadence is well below the 14 400 s threshold; see §6.5.

A5 Version rollback A signed-but-older firmware image is pushed to the fleet to revert devices to a vulnerable prior state. Endpoint signature verification accepts the image as long as the signing chain is still valid. OTA-Shield addresses this with rule R6, a version-monotonicity check against a per-BMS high-water mark; authorized downgrades are expressed as an explicit `rollback_window` in the RAT and handled by the rollback gate (§4.3.2).

The rules are numbered non-contiguously because R3 was reserved for a protocol-anomaly rule that we moved to the controller's manifest check. The data plane carries four mandatory rules (R2, R4, R5, R6) plus one optional rule (R1), together with one control-plane check. R1, R4, and R5 are the three threshold-bearing rules, R2 is identity-based, and R6 is monotonicity-based. R1 is enabled only when per-BMS update cadence is slower than $\tau_{R1} = 14\,400$ s; under faster cadences it is compiled out, leaving R2/R4/R5/R6 as the operational rule set.

This threat model also fixes where the detector can sit. OTA-Shield observes an OTA campaign only on a cleartext segment where it can recover per-packet source identity and OTA-header fields. Fig. 1 sets out the three placement paths this implies: a supported direct path after TLS/VPN termination, and two paths the evaluated build does not cover, a brokered path that collapses publisher source addresses and an encrypted or TCP-segmented path that hides the header. The two unsupported paths are characterized later as negative controls (E20, §7.2; E23, §6.5); we introduce them here so the scope is explicit before the design.

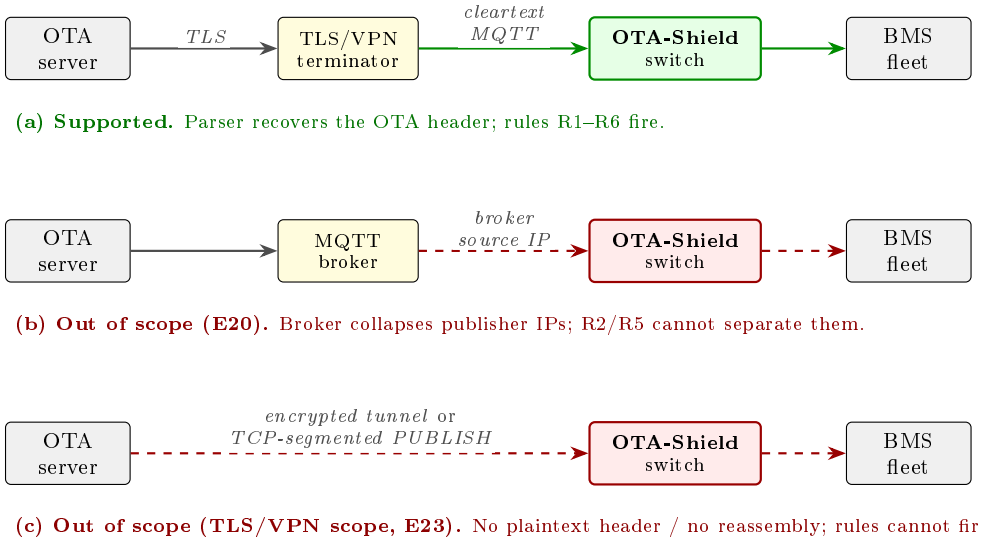
3. Related Work

We organize prior work by what each line of research can express, the assumptions it makes, and the limitation that leaves the OTA fleet-coordination problem open.

In-network security on programmable switches. Programmable-switch security work can enforce per-packet policy at line rate, but the published targets are volumetric or generic. Most P4/Tofino work targets volumetric DDoS: Poseidon compiles declarative defenses [13], Jaqen reaches 380 Gb/s with sketch-based mitigation [12], Patronum adds per-flow state [26], AlSabeh et al. run entropy-based DPI [27], and ACC-Turbo handles pulse-wave traffic [28]. Beyond DDoS, Kang et al. enforce context-aware enterprise access-control policies (BYOD) in the data plane without a control-plane round-trip [29], and Xing et al. mitigate network covert channels at line rate while preserving TCP performance [30]; both confirm that the programmability benefit reaches policy-enforcement and behavioral-anomaly problems beyond volumetric attacks. A parallel line pushes flow-level machine learning into the data plane [14, 15, 31]. The closest methodological neighbor is Bui et al. [16], which parses MQTT in P4 on the BMv2 software target and enforces topic-prefix authorization with 99.8% accuracy and sub-ms latency. Its assumption set differs from ours on platform, domain, threat model, policy model, stateful scope, and evaluation rigor; Table 2 marks the six axes. None of this prior work targets OTA firmware attacks on grid infrastructure.

Table 2
Axis-by-axis comparison with Bui et al. [16].

Axis	Bui et al.	OTA-Shield
Target platform	BMv2 software reference; no MAU budget or PHV layout constraints.	Tofino 1 hardware; reports measured layout against the 12-stage MAU budget, 48-byte digest quantum, and register restrictions (§4).
Target domain	Generic edge-IoT MQTT anomalies.	BESS firmware distribution on a grid-operator path (§2).
Threat model	Endpoint compromise and topic-ACL abuse.	Channel compromise with valid signing material and authorized topics, fleet-coordinated campaigns.
Policy model	Stateless topic-prefix authorization.	RAT with Gate A and Gate B arbitration (Alg. 1) over per-BMS rollout context.
Stateful scope	Session-level flow state.	Per-BMS version high-water (R6) and Bloom-gated fleet-fanout counter (R5).
Evaluation rigor	99.8% enforcement on software-timed BMv2.	Hardware latency (median ≈34.0 ms, p95 ≈46.6 ms), bootstrap F1 CIs over 20 stochastic trials, ablation (§6.4), adversarial-boundary sweep (§6.5), Suricata/Zeek baseline on identical traffic (§6.6).



(a) Supported. Parser recovers the OTA header; rules R1–R6 fire.

(b) Out of scope (E20). Broker collapses publisher IPs; R2/R5 cannot separate them.

(c) Out of scope (TLS/VPN scope, E23). No plaintext header / no reassembly; rules cannot fire.

Figure 1: Deployment scope, fixed by the threat model. OTA-Shield is supported only on a cleartext MQTT segment after TLS or VPN termination (a), where the parser recovers the OTA header and rules R1–R6 fire. Two paths are outside the evaluated build: (b) a brokered path, where the broker rewrites every PUBLISH to its own source IP and the source-based rules can no longer separate publishers (evaluated as a negative control in E20, §7.2); and (c) an encrypted tunnel that reaches the switch intact, or a PUBLISH split across TCP segments, where no single packet presents a parseable header and the data plane performs no reassembly (E23, §6.5). Dashed red paths mark unsupported placements, not demonstrated capability.

OTA firmware security and BESS prior work. This line of work secures the firmware after it arrives, which leaves fleet-wide coordination during distribution invisible. Firmware-update attacks on IoT devices are well documented [32, 33]; AoT shows replay and rollback on real devices [34]; SUIT [4] and Uptane [5] standardize cryptographic signing; and DDoSViT [35] defends the OTA channel from disruption. All of these verify firmware after delivery, so fleet-wide coordination during distribution is invisible to them. On the BESS side, Öhrström et al. give the first cloud-BESS attack reference model [2], Trevizan et al. catalogue BESS risks and controls [9], Culler et al. observe active scanning of exposed BMS interfaces [3], and GAN/ensemble detectors run on-device over telemetry [10]. The NERC RSTC white paper recommends in-network visibility as a compensating control for small DERs outside mandatory CIP scope; OTA-Shield fills that gap on switching silicon.

Threshold derivation and CPU-based IDS baselines. A CPU-based signature IDS has the throughput but not the arbitration needed for this problem. IDS anomaly thresholds are usually set from measured baseline distributions rather than expert heuristic [17, 18, 36]; we follow the same P99+$k\sigma$ pattern on a 90-minute on-testbed trace (§6). Suricata and Snort remain the reference open-source ICS IDS [8], and Suricata reaches tens of Gb/s via AF_PACKET [7], well above the OTA-channel loads here. The meaningful question is therefore not throughput but expressiveness: whether the rule language can capture stateful, RAT-aware arbitration. §6 answers that directly on identical traffic.

4. System Design

OTA-Shield splits detection between a Tofino 1 data plane that enforces per-packet rules inside the ASIC pipeline and a Python control plane that arbitrates ambiguous detections against a Rollout-Authorization Table (RAT). This subsection describes what each component does, why it is needed, and the constraint it addresses; the hardware realization follows in §5.

4.1. Architecture overview

Fig. 2 shows the split. The data plane carries the always-on, per-packet work: it parses the OTA header and evaluates the detection rules at forwarding speed, where a coordinated campaign's fanout shape is exposed. The control plane carries the slower, policy-aware work: it holds the rollout schedule and decides whether a flagged event belongs to an authorized rollout or to an attack. The split is needed because the campaign-versus-attack decision requires state (the rollout schedule) and matching logic that do not fit the per-packet ASIC budget, while the cheap campaign-shape signal must run on every packet without a control-plane round-trip.

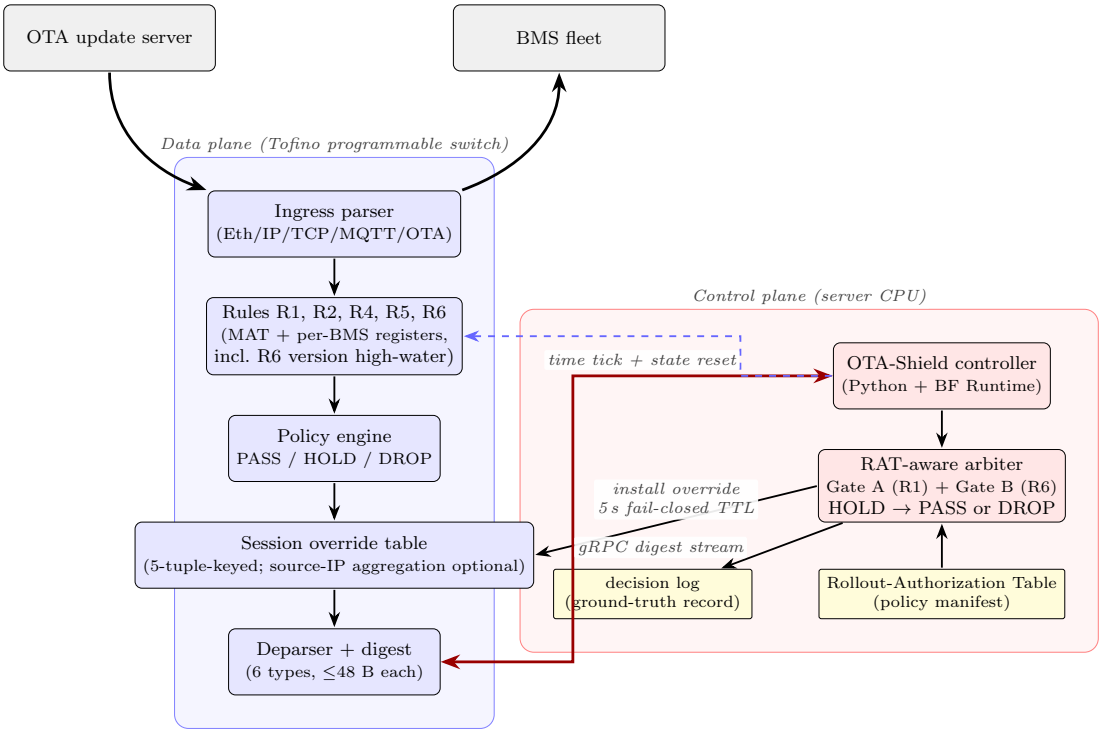


Figure 2: System architecture. The data plane (left) processes every packet through five detection rules (R1, R2, R4, R5, R6), backed by per-BMS state registers including an R6 version high-water mark; digests stream to the control plane (right) via gRPC; the RAT arbiter installs session overrides (5-tuple-keyed by default; a source-IP aggregation variant is preserved as a compile-time knob, see §6.7) with a 5 s fail-closed TTL. Gate A and Gate B (§4.3.1–4.3.2) run inside the arbiter.

4.2. Detection rules

Each rule answers one attack pattern from §2. R2 (unauthorized source) is an identity check on the firmware source IP against the operator's allow-list; its input is the IPv4 source address and its output is a terminal fire when the address is absent, implementing a fail-closed policy. R4 (oversize) compares the cumulative session byte count against the maximum legitimate firmware size; it is terminal because no authorized image exceeds that bound. R5 (fleet fanout) counts the distinct destination BMS addresses an *unauthorized* source reaches inside a 60-second window; its counter is gated on the unauthorized-source flag (`r2_fired`), so on the deployed build it charges only behind the terminal R2 drop and functions as a fanout telemetry signal rather than an independent verdict (§6.4). R6 (version rollback) compares the OTA-header version against a per-BMS high-water mark and fires when the observed version is strictly lower, which catches a signed-but-older image that endpoint signature verification accepts. R1 (rapid replay) measures the inter-update interval per BMS and fires when it falls below a cadence threshold; it is enabled only when per-BMS cadence is slower than τ_{R1}.

With compile-time thresholds τ_{R1}, τ_{R4}, τ_{R5}, the three threshold-bearing rules fire when

$$\text{R1 fires if } \Delta t_i < \tau_{R1},$$
$$\text{R4 fires if } B_i > \tau_{R4},$$
$$\text{R5 fires if } C_W > \tau_{R5},$$

where Δt_i is the inter-update interval for BMS i, B_i is the session byte count for session i, and C_W is the count of distinct destination BMS addresses inside the R5 window W. R2 fires when the source IP is not in the authorized-source allow-list, and the OTA-header sanity checks are handled by the control plane. R6 fires when the observed OTA-header version v_i is strictly less than the stored per-BMS high-water mark $v_i^\star$. R5's fleet-fanout count is charged only against unauthorized sources, so an authorized fleet rollout does not consume the detector's fanout budget; the implementation gates the count update on the R2 fire flag (§5).

4.3. Policy engine and RAT-aware arbiter

The policy engine splits enforcement into a fast data-plane verdict and a slower control-plane arbitration. In the data plane, the policy control block (Fig. 3) combines per-rule fire flags into a byte-wide action code. R2 (unauthorized source) drops the packet at the deparser. A behavioral rule fire (R1, R4, or R6) sets a `HOLD` action code and emits a `hold_digest` to the controller. The controller then decides: R4 stays terminal, while R1 and R6 go through the two-stage RAT arbiter (§4.3.1–4.3.2), which either installs a `session_allow` override (via Gate A or Gate B) or a `session_deny`. R5 is r2-gated:

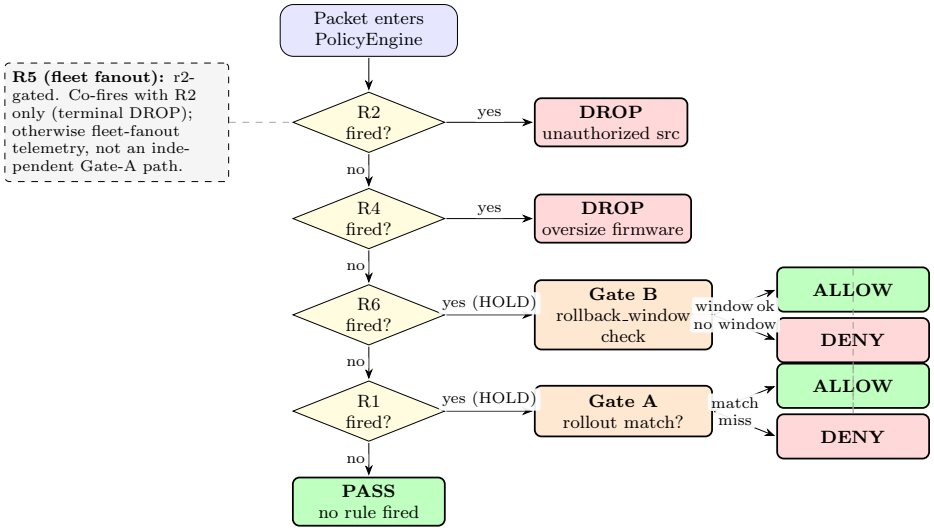


Figure 3: Policy decision flowchart. All rule fires emit a `hold_digest` to the controller. R2 (unauthorized source) and R4 (oversize payload) remain terminal DROP decisions. R1 (rapid replay) and R6 (version rollback) enter control-plane arbitration: Gate A demotes an R1 fire that matches an active authorized rollout to PASS (R5 cannot reach Gate A on the deployed build, its counter being r2-gated, §6.4), and Gate B demotes an R6 fire to PASS only when the active RAT entry carries an explicit `rollback_window` covering the observed version. Any other HOLD becomes a DROP override. Override entries expire after 5 s (fail-closed).

Table 3
Rollout-Authorization Table fields. The manifest is a per-rollout JSON object loaded by the controller; a HOLD packet passes only when every condition matches.

Field	Policy semantics
`authorized_source_ips`	Allow-list of IPv4 addresses permitted to originate firmware PUBLISHes for this rollout.
`target_bms_list`	Allow-list of destination BMS IPs within scope of the rollout.
`valid_window_start/end`	UTC wall-clock window during which the rollout is active.
`expected_payload_size_range`	Inclusive byte range for advertised firmware size.
`expected_firmware_version`	Version number reported in the OTA header for control-plane sanity check.
`max_concurrent_targets`	Enforced per-rollout cap on the number of concurrent active sessions a rollout may hold; the controller denies admission beyond it (0 = unlimited).
`rollback_window`	Optional {`min_version`, `max_version`} pair authorizing a rollback range for this rollout (see §4.3.2). Default-absent: R6 remains a terminal rule.

it co-fires with R2 (terminal DROP) and otherwise contributes a fleet-fanout flag to the digest rather than a distinct arbiter path. Packets that fire no rule pass with action code zero and no digest.

Packets with `action_code` $\neq 0$ generate a `hold_digest` (28 bytes, well under Tofino 1's 48-byte learn quantum) carrying the 5-tuple, session ID, BMS index, all five rule-fire flags, and the OTA version and size. The controller compares this digest against the RAT manifest (Table 3); a match installs a `session_allow` override, a miss installs `session_deny`. Both expire via a 5-second timer, giving fail-closed recovery if the controller stalls. Because the digest carries the BMS index and all five rule-fire flags, every detection is localized to a specific device and rule rather than reported only in aggregate, in the manner of localization-aware ICS detectors [37]; the per-strategy, per-rule breakdown this enables is reported for the mimicry sweep in §6.5.

4.3.1. Gate A: RAT-gated demotion of R1 fires

Gate A relaxes R1 (rapid replay) for packets that belong to an authorized rollout. Two stages run per HOLD digest. Stage one keeps R2 (unauthorized source) and R4 (oversize payload) terminal: any combination flagged as terminal installs `session_deny`. Stage two looks at R1 fires against the active RAT entries. R5 (fleet fanout) does not reach this gate on the deployed build: the data-plane R5 counter is gated on `r2_fired`, so R5 only fires alongside R2 (an unauthorized source), and stage one has already made that combination terminal.

Notation: r_1, r_2, r_4, r_5, r_6 are the rule-fire flags; s the source IP; d the destination BMS; v the OTA-header version. Gate A demotes an R1-only fire to PASS when $r_2 = r_4 = 0$ and a RAT entry R exists such that $s \in R.authorized\_source\_ips$, $d \in R.target\_bms\_list$, the current time lies inside $R.valid\_window$, the advertised size lies inside $R.expected\_payload\_size\_range$, and v matches $R.expected\_firmware\_version$ (or an

immediate neighbour, to tolerate staged major/minor bumps inside one rollout). The controller then installs a `session_allow` override instead of `session_deny`. On the benign workloads of §6.3 and §6.2, Gate A clears inter-update (R1) HOLDs on authorized staged, emergency, and migration rollouts to zero-drop outcomes, with no data-plane threshold retuned.

4.3.2. Gate B: RAT-gated demotion of R6 fires

Gate B relaxes R6 (version rollback) only when the RAT explicitly authorizes the downgrade. R6 follows the same shape as Gate A but through a parallel rollback gate. An R6-only fire ($r_6 = 1$, $r_2 = r_4 = 0$, no other terminal rule co-fired) is demoted to PASS only when a matching RAT entry R exists (same source/destination/window/size checks as Gate A), R carries an explicit `rollback_window` field, and v lies inside the range $[R.min\_version, R.max\_version]$ that the window advertises. If any check fails, R6 stays terminal and the controller installs `session_deny`.

The default is closed. A RAT entry that omits `rollback_window` leaves R6 as a terminal HOLD/-DROP rule, so a deployment that never authorizes a downgrade sees the same attack surface it had before. The gate exists to cover operator-authorized emergency rollbacks (for instance, an OEM recall that moves a fleet from version 48 back to version 47), while the data-plane monotonicity check still fires on every unauthorized downgrade.

Algorithm 1 Two-stage RAT-aware arbiter (Gate A and Gate B).

Require: digest D with flags r_1, r_2, r_4, r_5, r_6, source s, destination d, size b, version v; RAT manifest $\mathcal{R}$.

1: **if** $r_2 = 1$ **or** $r_4 = 1$ **then** ▷ terminal
2: **return** `session_deny`
3: **end if**
4: $R \leftarrow \text{MatchRAT}(\mathcal{R}, s, d, b, now)$
5: **if** $R = \varnothing$ **then**
6: **return** `session_deny`
7: **end if**
8: **if** $r_6 = 1$ **then** ▷ Gate B: default-closed
9: **if** *R.rollback_window* exists **and** $v \in R.rollback\_window$ **then**
10: **return** `session_allow`
11: **else**
12: **return** `session_deny`
13: **end if**
14: **end if**
15: **if** $r_1 = 1$ **then** ▷ Gate A (r_5 unreachable: gated on r_2)
16: **return** `session_allow`
17: **end if**
18: **return** `session_deny` ▷ unreachable under digest precondition

Gate A handles rollout-context demotion for R1; Gate B handles explicit rollback authorization for R6. MatchRAT checks source, destination, time window, and size range; it does not require exact version equality, which would reject legitimate staged major/minor bumps inside an active campaign. The final `session_deny` is unreachable on well-formed input: the data plane emits `hold_digest` only when $r_1 \vee r_2 \vee r_4 \vee r_5 \vee r_6 = 1$. The guards at lines 2–4 and 9 remove r_2, r_4, and r_6, and Gate A removes r_1. The one remaining case, an r_5-only fire, cannot occur on the gated build: the R5 counter is gated on `r2_fired`, so every r_5 fire also sets r_2 and is already caught by the terminal guard. It is retained as a defensive default for malformed or replayed digests. Every arbiter decision is logged, and all reported classification verdicts are read directly from this decision log.

5. Implementation

This section describes the Tofino 1 realization of the design: the ingress pipeline that evaluates the rules, and the controller that runs the arbiter.

5.1. P4 ingress pipeline

The ingress pipeline (Fig. 4) comprises a protocol-aware parser and nine MAU-stage control blocks, within the 12-stage Tofino 1 budget.

Parser. The ingress parser identifies MQTT traffic on TCP port 1883. For MQTT PUBLISH packets (`mtype`=3), it extracts the variable-length remaining-length field (1–4 bytes), a 32-byte null-padded topic slot, a QoS-1 packet identifier, and the first 20 bytes of the payload as the OTA header. The OTA header carries a 4-byte magic (`0x4F544153` = "OTAS"), a 4-byte firmware version, a 4-byte advertised size, and an 8-byte hash hint. All metadata fields are byte-aligned (`bit<8>`) to satisfy Tofino 1's PHV allocator constraints.

Session manager. A CRC32 hash of the 5-tuple produces a 16-bit session index into two 65 536-entry registers: `session_bytes_reg` (cumulative TCP payload bytes) and `session_first_ts_reg` (first-seen timestamp with a non-zero sentinel). On TCP FIN or RST, a read-and-clear action atomically returns the session summary and resets the slot.

Fleet monitor (R5). Three independent Bloom filters (1024 entries each, CRC32 / CRC16 / CCITT hashes with salt bytes) track distinct destination BMS addresses within a 60-second tumbling window. A single `RegisterAction` on a 16-bit counter increments on Bloom miss and returns the current count. A range-match table fires R5 when the count exceeds the threshold (compile-time constant, default 4). The controller clears all three Bloom registers and the counter every 60 seconds. Bloom inserts and the count update are gated on `r2_fired==1` so authorized fleet rollouts do not consume Bloom slots; only unauthorized fanout charges against the detector. This Bloom-sizing assumption is revisited in §7.2 for fleet-scale deployments.

Secondary rules (R1, R4). R1 uses a 256-slot 16-bit register indexed by BMS number. The stateful

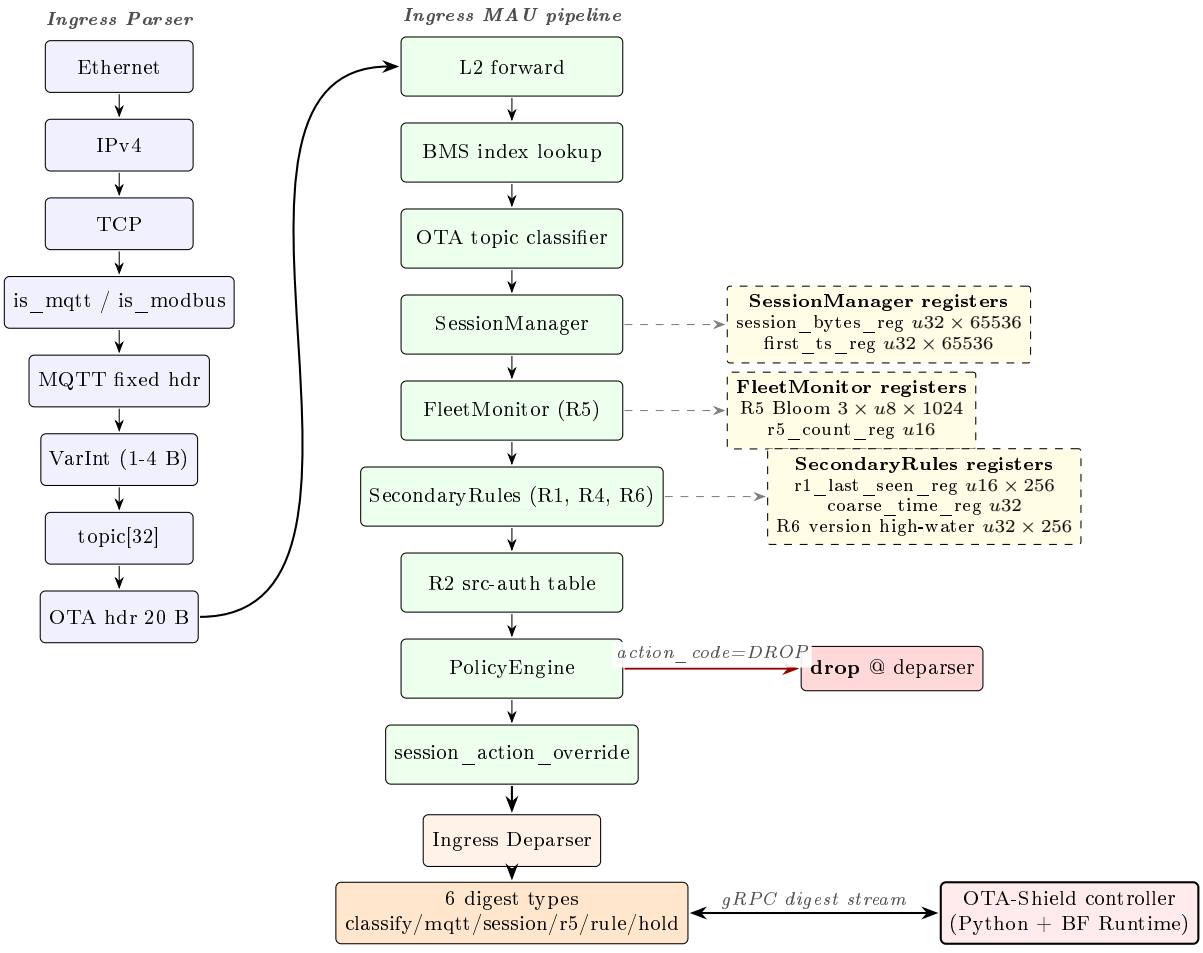


Figure 4: P4 ingress pipeline. Dashed arrows connect MAU control blocks to their stateful registers. Topic and OTA header use fixed-size slots; other parameter combinations are discussed in §6.7.

ALU computes `coarse_time_sec_lo - last_seen` as the inter-update delta; a range-match table fires R1 when the delta is below the threshold (compile-time constant, derived from the E7 baseline as 14 400 s = 4 h; see §6.2). R4 compares the upper 16 bits of `session_bytes_val` against a range threshold (compile-time constant, derived from E7 as 32 units of 64 KiB = 2 MiB). R2 is an exact-match table on `ipv4.src_addr`: entries for authorized OTA sources are installed by the controller at startup from the RAT manifest. The default action fires R2, implementing a fail-closed policy.

Version rollback (R6). R6 keeps a per-BMS firmware-version high-water mark in a 256-slot 32-bit register indexed by BMS number. The stateful ALU compares the incoming OTA-header version against the stored mark, fires R6 when the observed version is strictly below it, and raises the slot to the maximum of the two values. There is no threshold to tune: the check is version-monotonic and catches signed-but-older firmware that endpoint signature verification lets through. An R6 fire emits a `hold_digest`, and the control plane decides. Gate B consults the RAT and clears the fire only when an authorized rollback entry is active.

Compile-time thresholds. All thresholds are compile-time constants (range-match table entries) rather than runtime registers, for two reasons. First, a runtime register comparison would need two runtime operands in the same MAU action, which the Tofino 1 gateway's predicate-width constraint disallows. Second, the thresholds are derived once per deployment from an E7-equivalent baseline (§6.2) and are not intended to drift; pinning them at compile time makes their provenance auditable and excludes a silent-override vector on the control plane. Re-deriving thresholds for a new deployment profile is a P4 recompile (minutes), not a runtime reconfiguration.

5.2. Controller

The controller attaches to the switch over local-loopback gRPC. It installs R2 allow-list entries and Bloom-clear timers at startup, receives each `hold_digest`, and runs the two-stage arbiter

of §4.3 on every HOLD. A match installs a `session_allow` override on the 5-tuple-keyed `session_action_override` table; a miss installs `session_deny`. Both expire after a 5-second fail-closed TTL. The controller loads the RAT manifest at startup and on hot-reload, verifies its ed25519 signature when run under `-require-signed-rat`, and retains the last-known-good manifest when a reload fails verification.

6. Evaluation

This section answers five research questions about OTA-Shield with the testbed and methodology described in §6.1–6.2.

Main result map. The evaluation has five load-bearing results. E8 reports the headline designed-family detection result; E12 tests whether authorized rollouts are admitted without false drops; E19 and E19′ isolate the rollback rule and its first-contact limitation; E21 measures adaptive mimicry as the main adversarial blind zone; and E10 compares OTA-Shield against CPU-based IDS baselines on identical traffic. The remaining experiments are support, boundary, or artifact-provenance checks, and the *Role* column of Table 7 marks each one.

Table 4
Contribution-to-evidence map. Each contribution is tied to the experiment that carries it and to the boundary that bounds it.

Contribution	Evidence	Boundary / limitation
Bounded attack detection	E8	designed family only
Authorized rollout admission	E12	RAT correctness required
Signed rollback detection	E19, E19′	first-contact cold start
CPU-IDS comparison	E10	CPU port matches detection quality; the difference is resource placement, not accuracy
Deployment-boundary characterization	E20, E21, E23, E18	brokered MQTT, mimicry, TCP segmentation, parser variation
Capacity and latency	E11, E13, E14	generator/digest-channel bound, not switch line-rate

- **RQ1.** Does the detector classify attack and benign OTA events correctly on the reference testbed? (E1, E2, E6, E8, E12, E12b.)
- **RQ2.** Is the RAT-aware arbiter necessary, and is R6 the sole detector of unauthorized rollback? (E4, E19, E19′.)
- **RQ3.** How does each rule degrade under adversarial mimicry, threshold sweeps, and long-horizon slow-cadence traffic? (E9, E21, E7b.)
- **RQ4.** How does OTA-Shield compare to CPU-based IDS baselines on identical traffic? (E10 variants i–vii.)
- **RQ5.** What latency, throughput, capacity, and portability costs does the deployment pay? (E14, E11, E13, E18.)

Every result below is measured on one deployed configuration, summarized in Table 5; Table 6 maps each attack pattern to the rule that answers it and to that rule's measured blind zone.

Table 5
Final evaluated build. All results in this section are measured on this single deployed configuration.

Component	Final setting
R5 gating	r2-gated (counts unauthorized-source fanout only)
Override table	5-tuple-keyed default; source-IP aggregation optional
Per-source hold gate	inert (arms on R5 only, which is r2-shadowed)
OTA profile	cleartext MQTT reference profile (TCP/1883)
Parser	32-byte null-padded topic slot + 20-byte OTA header
Fleet	50 BMS emulated endpoints

Table 7 indexes every experiment to the RQ it supports and to the subsection in which it is reported.

6.1. Testbed

The testbed (Fig. 5) consists of two commodity servers bridged by a programmable switch. One server is the OTA update server, publishing MQTT firmware messages. The other emulates a 50-unit BMS fleet using network-layer aliases, so every unit appears on the wire as a distinct endpoint. Both servers carry 25 Gb/s network interfaces and connect to the switch through RS-FEC links. The switch runs BF-SDE 9.13.2, with the OTA-Shield P4 pipeline loaded and the Python controller attached over local-loopback gRPC.

Traffic generation is scripted: each scenario emits MQTT PUBLISH packets carrying the 20-byte OTA header, records a wall-clock send timestamp and a ground-truth label (LEGIT or ATTACK), and writes the record to a per-trial log. The controller writes every policy decision to a second log. We join the two logs by 4-tuple (`src_ip`, `dst_ip`, `src_port`, `dst_port`) so each labelled event inherits the detector's verdict.

6.2. Methodology

Trial independence. Between trials the sweep driver signals the controller, which re-seeds all R1 last-seen register slots, clears the R5 Bloom filters and counter, and waits 70 s for the R5 window to expire. Each trial starts from identical detector state.

Ground-truth labeling. Labels are assigned by the scenario generator, not by the detector. Events with no matching controller decision or digest are counted as NO_DECISION rather than silently dropped, preventing digest-transport loss from inflating reported metrics.

Table 6
Rule/attack coverage on the final evaluated build. Each rule answers one attack pattern; the rightmost column states the measured blind zone.

Attack pattern	Rule	Detected in final build?	Main evidence	Known blind zone
Unauthorized firmware source	R2	Yes, terminal DROP	E1, E8	brokered-MQTT source-IP collapse (E20)
Oversize firmware image	R4	Yes, $\geq$2.0625 MiB	E6, E9	2.0–2.0625 MiB quantization dead zone
Coordinated fleet fanout	R5	Telemetry behind R2, not independent	E9, §6.4	authorized-source fanout (out of scope)
Signed version rollback	R6	Yes	E19, E19', E7b	first-contact cold-start
Rapid replay	R1	Cadence-dependent, non-central	E1, E9	inter-update interval $> \tau_{R1}$

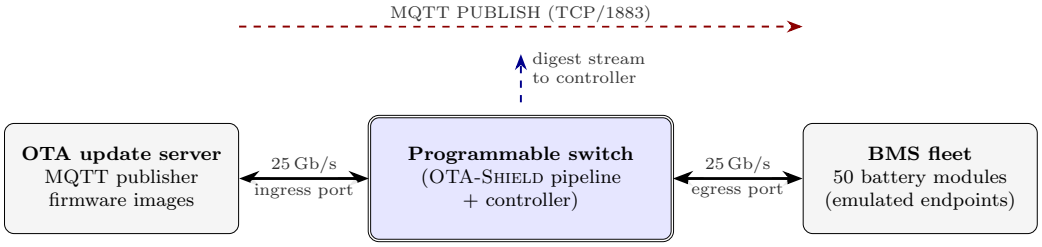


Figure 5: Testbed topology. An OTA update server and a 50-unit BMS fleet connect to a programmable switch through data-plane links configured at 25 Gb/s with RS-FEC (link capability; the experiments in this paper do not offer traffic at that rate). A separate management LAN carries control-plane gRPC traffic.

Threshold derivation versus evaluation (E7). Baseline traffic from the long-run benign trace is used only to derive R5, R1, and R4 thresholds; it is never used to score detection metrics. All reported precision, recall, F_1, and latency numbers come from separately generated evaluation scenarios (E1, E4, E6, E8, E9, E10, E11) that the thresholds have not been exposed to. A 2709-event Poisson baseline trace (≈90 min, mean IAT 2 s) of authorized PUBLISHes reaches random BMSes with firmware sizes drawn uniformly from 256 KiB–2 MiB. From this baseline we compute three distributions: (a) distinct-BMS count per 60-second window (R5 metric), (b) per-BMS inter-rollout interval (R1 metric), and (c) firmware size (R4 metric). The P99.9 tail of the distinct-BMS count is 30.0 (legitimate fleet rollouts reach at most ≈30.0 distinct BMS targets per 60-second window at P99.9), the P0.1 tail of the inter-rollout interval is 0.1 s (R1 minimum interval 1 s), and the P99.9 firmware size is 1995371 B (R4 bound 2095947 B). R4 is pinned directly at the E7-derived ≈2 MiB bound. R5 is a compile-time constant set to 4 (fires when distinct-BMS count exceeds 4 per 60-s window), substantially below E7's P99.9 of 30.0; R5 is intentionally tighter than the baseline tail to target attacks reaching $\geq$5 distinct BMS targets, a deliberate design choice for the 50-unit testbed scale, not a threshold derived from the P99.9 tail. R1 is pinned looser (14,400 s vs. E7's P0.1) to reflect the update cadence of scheduled campaigns rather than the minimum legal inter-message gap.

Deterministic versus stochastic experiments. Experiments with fixed seeds (E1, E4, E5, E6) function as correctness tests; repeating a deterministic computation is not statistical sampling, so they are reported without CIs. E8 introduces random seed, BMS ordering, and inter-arrival jitter per trial; bootstrap 95% CIs on E8 are the paper's headline statistical result. E19' (20 trials, 800 events: 460 rollback attacks and 340 legitimate events) randomizes per-BMS version ordering and inter-arrival jitter under a fixed top-level seed. On the deployed gated build it yields zero benign false positives (precision 1.000, Wilson 95% CI [0.990, 1.000]); the genuine between-trial variance now lives in *recall*, because a stochastic share of the rollbacks are first-contact events that R6 structurally cannot catch on the version it has never seen (§6.4). We therefore report a strict recall that counts those first-contact misses (0.857, Wilson 95% CI [0.822, 0.886]) alongside the lenient figure that excludes them (0.970). Because OTA-Shield's rules apply hard compile-time thresholds with no tunable per-event score, a swept precision–recall curve is degenerate (precision stays at 1.000 across the operating range), so we report precision, recall, and F_β at the deployed fixed-threshold operating point and present the cross-detector trade-off as operating points on the precision–recall plane (Fig. 13), the appropriate presentation under the class skew here [38]. Where an experiment observes zero false positives or negatives we report a Clopper–Pearson rule-of-three one-sided 95% upper bound on the error rate ($\leq$ 0.50% for E8, $\leq$ 0.67% for E12, i.e. at most 0.50% and 0.67% of events could be mislabelled at the 95% confidence level given the observed 0/N count); because these are correctness runs rather than independent significance tests, a strict Bonferroni correction is inappropriate. For E8 precision specifically, FP = 0 across all 20 trials and 600 benign events; the bootstrap CI compresses to a degenerate [1.000, 1.000] and is replaced by the Clopper–Pearson

one-sided 95% upper bound on FP rate of ≤0.50% (= 3/600, identical to the rule-of-three bound).

Train/test discipline and capture duration. OTA-Shield has no learned parameters. The rules are fixed thresholds and the RAT is operator-supplied, so there is no train/test split to leak across. We instead guard against the inflated-score failure mode that afflicts learned ICS detectors [39] in two ways. First, the benign test set contains operationally *unusual but legitimate* events: emergency hotfix re-pushes, mid-rollout source migration, delayed-retry bursts, and an authorized fleet rollback (E12). A PASS verdict is therefore earned against traffic that superficially resembles an attack, not against quiescent background. Second, the adversarial sweeps (E9, E21, E23, E20) probe attack variants the rules were not hand-tuned against, including sub-threshold mimicry and TCP segmentation, so the reported coverage is on unseen behavior. The zero-false-positive bounds above are tied to event counts rather than wall-clock duration; for reference the benign captures span roughly 3.5 min (E8, 600 events) and 25 min (E12, 450 events) plus the longer signed-manifest soak of E12b. A multi-hour production soak in the manner of Caselli et al. [40] is future work; at the emulated testbed's compressed cadence we report the event-count bound as primary.

Bootstrap for stochastic CIs. Stochastic experiments report 95% confidence intervals from a trial-level percentile bootstrap: per-trial point estimates (precision, recall, F_1) are resampled with replacement over the n trials, using $B = 2\,000$ replicates, and the 2.5th and 97.5th percentile of the resample distribution are reported as the CI bounds. The confidence level is $\alpha = 0.05$. When all trials yield the same point estimate the resulting distribution is degenerate; we report the point in that case and instead quote the Clopper–Pearson one-sided bound described above.

6.3. RQ1: Correctness on the reference testbed

Finding: rules fire only where they should, and the arbiter never drops an authorized rollout, including under stochastic perturbation and signed-manifest tampering.

The first block of experiments checks four things: that each rule fires exactly on its designed trigger, that the arbiter returns the intended verdict on every event, that stochastic perturbation does not collapse those guarantees, and that the benign dual (the arbiter must not drop authorized rollouts) holds end-to-end.

E1: attack detection. Each trial sends 30 baseline legitimate PUBLISHes, 30 rapid replays to the same BMSes (R1 fires; the replay source is authorized, so R2 does not fire and R5's r2-gated counter never runs), and 30 unauthorized-source PUBLISHes (R2 fires), 90 events. Across 5 trials, every trial produces precision 1.000, recall 1.000, $F_1 = 1.000$. Each rule fires correctly on its designed trigger and the RAT arbiter classifies every event as intended.

E2: false-positive baseline. 50 authorized PUBLISHes per trial (all LEGIT), 5 trials. No false positives.

E6: R4 oversize detection. A single MQTT session pushes 2.125 MiB of TCP payload (above the 2 MiB threshold). R4 fires deterministically on all 3 trials.

Stochastic detection performance (E8). E8 mirrors the E1 scenario but introduces realistic stochasticity: random BMS ordering, Poisson inter-arrival times (mean 100 ms), and ephemeral source ports drawn uniformly from 49152–65535. Each of 20 trials uses a distinct random seed, producing independent draws. Across the 20 trials we observe precision = 1.000 (FP = 0/600 benign events; CP 95% UB on FP rate ≤0.50%; bootstrap CI degenerate), recall = 0.992 [0.986, 0.998], $F_1 = 0.996$ [0.993, 0.999] (trial-level percentile bootstrap 95% CIs, $B = 2\,000$ resamples). The 20-trial sample size was chosen so that the bootstrap CI half-width for F_1 is below 0.01, verified by plotting CI width as a function of trial count from 5 to 20; the width plateaus by trial 15. Per-trial F_1 dispersion is tight (interquartile range within the CI, Fig. 7), consistent with the low missed-event rate on this workload; the pooled confusion matrix is Fig. 6. E8's attack mix inherits E1's composition (30 R1 rapid-replay events from the authorized source and 30 R2 unauthorized-source events per trial, plus their benign baseline); R4 oversize, R5 fanout from an unauthorized source, and R6 rollback are exercised in separate experiments (E6, E9, E21, E19). The headline F_1 therefore covers the R1 rapid-replay and R2 unauthorized-source families at stochastic scale on the 50-unit testbed; fleet-scaling precision at 200 BMS is reported separately in E1′/E8′ (Table 11, §6.7).

Benign operational rollout (E12). E12 answers the dual of the attack experiments: does the arbiter clear rollouts that an operator is entitled to run? The workload drives six legitimate patterns through Gate A and Gate B (§4.3.1–4.3.2). The backbone is a five-wave staged rollout (ten BMSes per wave, 30 s between waves, 65 s pause to clear the R5 window), and on top of that: an emergency hotfix push that re-hits the same BMS inside R1's 14 400 s window; two migration sub-scenarios that switch the authorized source mid-rollout; a delayed retry burst that compresses a staged wave into a few seconds; a rollback preamble that primes each BMS's version high-water mark; and a benign authorized rollback that downgrades the fleet by one version under an explicit `rollback_window` RAT entry. Every event is labelled LEGIT, so PASS is the correct verdict throughout.

Table 7
Experiment roadmap indexed by RQ. The *Role* column tiers each experiment as PRIMARY (a load-bearing claim: detection, admission, rollback, mimicry blind zone, or baseline), BOUNDARY (a scope or blind-zone limit), SUPPORT (rule correctness, capacity, latency, or cost), or ARTIFACT (manifest-lifecycle and build-provenance evidence). Rows marked † use the trial-level percentile bootstrap of §6.2; rows marked ‡ use Wilson 95% CIs (or the Clopper–Pearson rule-of-three one-sided bound on zero-event cells); remaining rows are deterministic point counts. Deterministic trials (E1, E4, E6, E10) yield identical outcomes by construction; E11, E12, E13, E21, E18 report per-event or per-scenario counts rather than classification F_1.

Exp.	Purpose	Type	Trials	RQ	Role	Headline
E1	Correctness (attack detection)	deterministic	5	RQ1	Support	$F_1 = 1.000$
E2‡	FP baseline	deterministic	5	RQ1	Support	0 FP
E4	RAT arbiter ablation	deterministic	5	RQ2	Support	$F_1 = 1.000$
E6	Oversized firmware (R4)	deterministic	3	RQ1	Support	$F_1 = 1.000$
E8†	Stochastic IID trials	stochastic	20	RQ1	Primary	$F_1 = 0.996$ [0.993, 0.999]
E9	Adversarial evasion sweeps	deterministic	3×4	RQ3	Boundary	boundary maps
E10	Suricata/Zeek baseline	deterministic	1 (PCAP replay)	RQ4	Primary	see §6.6
E11	Controller digest-channel rate	stress run	1	RQ5	Support	0.09% aggregate loss
E12‡	Benign rollout stress	stress run	3	RQ1	Primary	0 DROP on 450 legit events
E13	Override-table capacity	stress run	1	RQ5	Support	sat. at 4–8 s
E14†	Latency stage breakdown	post-hoc	on E8	RQ5	Support	stage 1 p95 41.8 ms
E21	Per-strategy mimicry	deterministic	30	RQ3	Primary	1.7% coverage (§6.5)
E18	Reference-channel portability	deterministic	1	RQ5	Boundary	4/8 variants parse
E19	R6 rollback ablation	deterministic	3	RQ2	Primary	$F_1 = 1.000$ with R6; collapses without R6 (§6.4)
E12b‡	Signed-manifest fail-safe	stress run	20	RQ1	Artifact	last-known-good (LKG) pass-rate = 100% under sig-t
E7b‡	Long-horizon slow cadence	7.9 h trace	1	RQ3	Support	R6 recall 1.000, precision 1.000 (§6.5)
E23	TCP-segmentation evasion	deterministic	250	RQ3	Boundary	header-split evades all rules (§6.5)
E20	Brokered-MQTT topology	deterministic	80	RQ3	Boundary	IP-collapse drops benign fleet (§7.2)

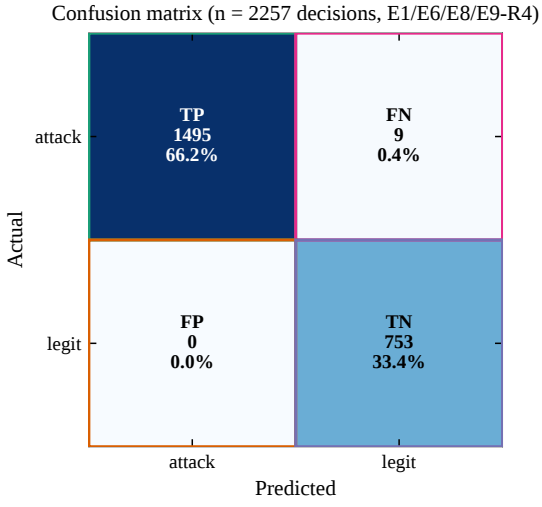


Figure 6: Pooled confusion matrix across the four scoring experiments (E1 attack detection, E6 oversize, E8 stochastic, E9 R4 evasion; $n = 2257$ decisions). Zero false positives and nine false negatives; the nine FNs come from the stochastic trials of E8 and match its recall of 0.992. E4 is an arbiter ablation (RAT disabled), not a detector-scoring run, so it is excluded from the pooled detector matrix.

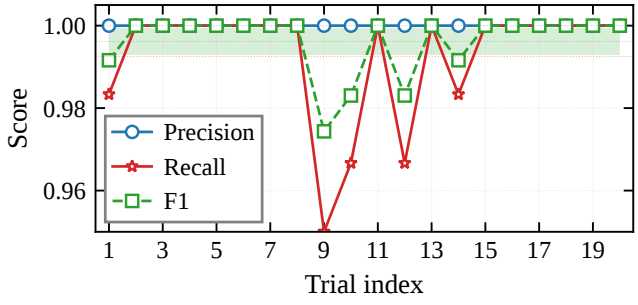


Figure 7: Per-trial precision, recall, and F_1 across the 20 stochastic E8 trials, with the F_1 bootstrap 95% CI band shaded. The narrow dispersion shows that stochastic performance stays close to the CI interval reported for E8 in Table 7.

Across 3 trials (Fig. 8) the arbiter produced 0 DROPs on 450 legitimate events (100.00% PASS): 150 silent PASSes (no rule fired) and 300 HOLD-only demotions, the latter being packets that tripped a data-plane rule and were cleared by the control-plane gate. On the deployed gated build the per-rule tally is simple: 270 events fire R1 (a re-push inside R1's window) and are demoted to PASS by Gate A, and 30 authorized rollbacks fire R1+R6 and are demoted by Gate B under an explicit `rollback_window` RAT entry. No legitimate event fires R5 (it is r2-gated, §6.4), so the benign rollouts produce no false positives. The three trials are a reproducibility probe rather than a statistical sample (the scenario is deterministic by construction), so the 0/450 FP count is reported with a Clopper–Pearson one-sided 95% upper bound of 0.66% (at most that fraction of legitimate events could be mislabelled at the 95% confidence level) rather than as an empirical point estimate. No rule-group produced a false positive.

Signed-manifest fail-safe (E12b). E12b validates the signed-manifest path end-to-end under controlled tampering. Each of 20 trials has two phases: Phase A runs a benign rollout under a valid ed25519-signed RAT; Phase B injects a single-byte flip on the detached signature file, forcing the controller's hot-reload to reject

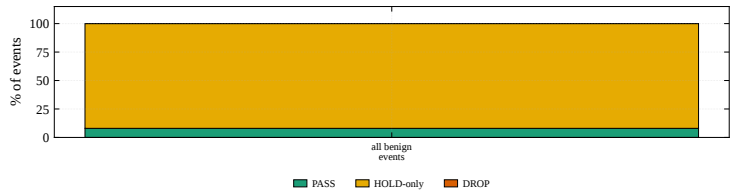


Figure 8: E12 aggregate outcome across 450 legitimate events (3 trials) on the deployed gated build. Classify-only silent PASS (no rule fired) covers 150 events; the remaining 300 events trip R1 (and R6 on authorized rollbacks) and are demoted to PASS by Gate A (270) or Gate B (30). No legitimate event fires R5: its fanout counter is gated on the unauthorized-source flag (§6.4), so authorized rollouts never reach it. Zero legitimate events are dropped.

the update and continue serving the LKG manifest. Across Phase A the arbiter processes 3000 events with 0 DROPs (FP-rate 0.0000); across Phase B the LKG manifest holds 160/160 events at PASS, so the tampered reload never corrupts policy state. The stale-RAT precision failure mode (expired `valid_window`, no replacement installed) is a distinct lifecycle condition discussed in the RAT lifecycle matrix (§7.7).

6.4. RQ2: Arbiter and R6 necessity

Finding: the RAT arbiter is necessary for admitting authorized operations, not for attack precision. It clears the authorized rollouts that trip the replay rule R1 (E12, zero benign-rollout false positives) and the operator rollbacks that R6 would otherwise drop (E19, Gate B), and R6 is the sole detector of unauthorized rollback (removing it makes rollback attacks invisible, E19). Disabling the arbiter leaves attack precision unchanged (E4), because the data-plane terminal rules carry attack detection.

RQ2 establishes the arbiter's necessity on the operations it admits, the benign rollouts of E12 (§6.3) and the operator rollbacks of E19 below, and isolates R6 as the sole rollback detector. The attack-side sanity check (E4) comes last.

R5 gating and the deployed-build verdict. On the deployed build the fleet-fanout counter (R5) is gated on `r2_fired`, and R2 is a terminal DROP that the policy engine resolves before the R5 branch. R5 therefore never produces an independent verdict over R2: it contributes fleet-fanout *telemetry* rather than a distinct line-rate action. The per-source `hold_armed_reg` gate arms only on R5, so behind the terminal R2 drop it does not arm on any reachable input and is inert (§7.2). The operational consequence is that an authorized rollout which trips the per-BMS replay rule (R1) or the rollback rule (R6) reaches the arbiter and is admitted, rather than being dropped at line rate before arbitration. The defended space is therefore unauthorized-source campaigns and unauthorized rollbacks, not arbitrary coordinated fanout from an already-authorized source; that blind spot is stated in §7. We keep the gating because it removes benign-rollout false positives, at the cost of R5's marginal independent detection, a trade we judge correct for an OT deployment where a single false quarantine of a legitimate rollout is operationally expensive.

This is the configuration under which E12, E19, and E19′ are measured: E12 (3 trials, 450 legitimate events) reports zero benign-rollout false positives, and E19′ (20 stochastic trials) reports precision 1.000 (Wilson 95% CI $[0.990, 1.000]$) and strict recall 0.857 (Wilson 95% CI $[0.822, 0.886]$). The strict recall counts first-contact rollbacks, which R6 structurally cannot catch because a first contact to a BMS has no stored version high-water mark, as misses (54 of 66 missed attacks); lenient recall excluding them is 0.970. We report the strict figure (rationale in §6.4); R6's first-contact blind spot is a scope limitation addressed in §7, with a baseline-warmup or cross-source version prior as the natural mitigation.

Rollback ablation (E19). A targeted ablation confirms that R6 is the sole rollback detector and that disabling it cannot be compensated by R1 alone. With R6 enabled, the arbiter produces 1.000 recall and 1.000 precision over 60 unauthorized rollback events across 3 trials (median end-to-end latency 24.0 ms). With R6 disabled, detection collapses to 0.0%: across 60 rollback ground-truth events in 3 ablation trials, the detector produced 0 DROPs. R1 alone cannot catch a v48→v47 regression because R1 is a sliding-window inter-update test, not a version-monotonicity test; only R6 enforces the high-water mark.

Stochastic rollback (E19′). E19′ runs the unauthorized-rollback experiment with three axes resampled per trial under independent RNG seeds: rollback delta $\sim$ Uniform(9, 16), benign-to-attack event ratio $\sim$ Uniform(0.3, 0.7), and inter-packet gap $= 20$ ms $+ \mathcal{N}(0, 15$ ms) clipped to $[10, 200]$ ms. Across 20 trials on the deployed gated build the detector reports precision = 1.000 [0.990, 1.000] (Wilson 95% CI; zero benign false positives across 340 legitimate events, Clopper–Pearson one-sided 95% upper bound 0.88%) and strict recall = 0.857 [0.822, 0.886], where strict recall counts a first-contact no-decision as a miss. Of the 460 rollback attacks the detector drops 394; the 66 misses split into 12 reachable misses and 54 first-contact rollbacks that R6 structurally cannot

catch, because a first contact to a BMS leaves no stored version high-water mark to compare against. Excluding those cold-start cases gives a lenient recall of 0.970 [0.949, 0.983]; we headline the strict figure, in line with evaluation-integrity guidance for security detectors [41, 42]. Because a false quarantine of a legitimate rollout is operationally more costly than a missed cold-start rollback in this setting, we also summarise the operating point with a precision-weighted F_β ($\beta = 0.5$, van Rijsbergen [43]): $F_{0.5} = 0.968$ at strict recall and 0.994 at lenient recall. The strict recall reflects R6's first-contact blind spot rather than a detection regression. Controller-side (stage-2) detection latency: 10.7 ms median (p95 15.0 ms); adding the E14 stage-1 measurement (≈29.5 ms) gives end-to-end ≈40.2 ms. Per-trial outcomes and the first-contact recall taxonomy are in Appendix B.

Arbiter ablation (E4), a sanity check. For completeness we confirm the converse: the arbiter is not needed for attack precision. Running the E1 workload against a controller launched with `-disable-rat` (which forces every HOLD to DROP) leaves precision at 1.000 and recall at 1.0, a change of Δ precision $= 0.0$ pp over 5 trials. Every drop is a terminal R1/R2 fire, and no decision is a HOLD the RAT clears, because R5 is gated on `r2_fired` and never charges on an authorized rollout. The arbiter's value is in admission (E12, E19), not attack precision.

6.5. RQ3: Adversarial robustness

Finding: adversaries who shape traffic around each rule's threshold can escape the data plane: 30/1770 (1.7%) of mimicry events are dropped, and the residual blind zones are reported per rule below (Fig. 9).

RQ3 asks how each rule degrades when an adversary is free to choose attack parameters near the rule's detection boundary or to slow-play an attack across a realistic deployment horizon.

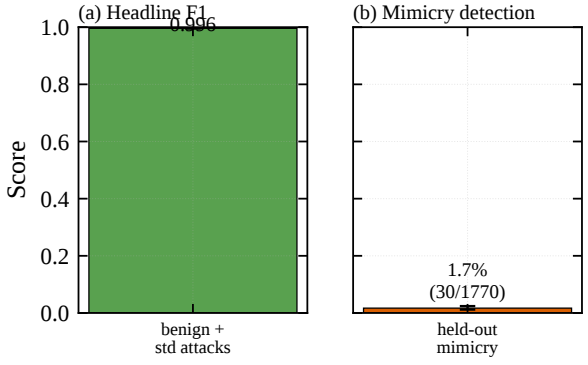


Figure 9: The two headline numbers, shown co-equal on a common $[0, 1]$ scale: (a) naive fleet-fanout detection $F_1 = 0.996$ (E8) versus (b) adaptive-mimicry coverage 1.7% (E21). The headline applies to the designed naive family; an adversary who reshapes cadence faces the right-hand number.

Adversarial evasion sweeps (E9). E9 probes the detection boundaries of R1, R4, and R5 by varying the attack parameter across the threshold edge. *R5 sweep:* fanout sizes {3, 4, 5, 6, 7, 10, 15, 20} with disjoint BMS ranges and 65-second pauses; the transition from no-fire to fire occurs at fanout 5, matching the compile-time threshold of 4 (count $> 4 \Rightarrow$ fire). Above the threshold, P(R5 fires) grows as $(N - 4)/N$ (Fig. 10), confirming the rule's intrinsic count > 4 boundary. This sweep characterizes R5's counter logic directly; on the deployed build that counter is gated on `r2_fired`, so an *authorized* fleet rollout never increments it and never fires R5 (§6.4). The authorized-rollout PASS rate is therefore 1.0 trivially: there is no R5 fire for the arbiter to demote, not because the arbiter clears an R5 HOLD. *R4 sweep:* flow sizes {1.5, 1.99, 2.0, 2.01, 4.0, 8.0} MiB; R4 fires only at and above 2.0625 MiB, the first size whose upper-16-bit field reaches the range-match threshold (the data-plane compare is `session_bytes[31:16]` ≥ 33, i.e. 33×64 KiB). The nominal oversize threshold is 2 MiB (32×64 KiB), but the 64 KiB quantization of the upper-16-bit key means a flow does not trip until it crosses 2.0625 MiB; the 2.0 and 2.01 MiB sweep points fall inside this $[2.0, 2.0625)$ MiB dead zone and do not fire. This is a documented limitation of the range-match encoding. *R1 sweep:* inter-replay intervals {10, 14399, 14400, 14401} seconds; R1 fires at intervals ≤ 14399 s and does not fire at ≥ 14400 s, confirming the exact boundary. The 16-bit wrap-around creates a theoretical false-positive window at 18.2-hour multiples (§7).

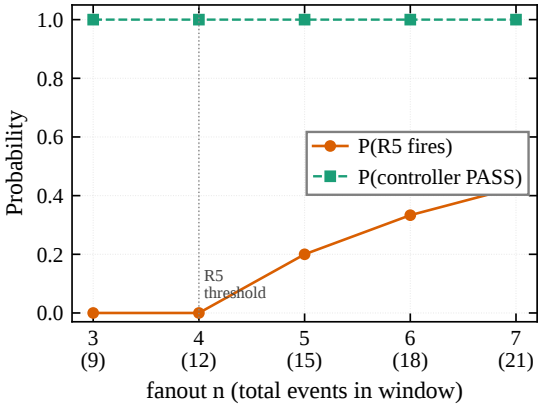


Figure 10: R5 sensitivity as a function of fanout size: P(R5 fires) grows linearly above threshold 4, characterizing the rule's intrinsic counter. On the deployed build R5's counter is r2-gated, so an authorized fanout never fires R5 (§6.4); the authorized-rollout PASS rate is 1.0 trivially, with no R5 fire to demote.

R1 behavior at high traffic rates. Under E7's traffic profile (one PUBLISH per 2 s mean across 50 BMSes), the mean per-BMS revisit interval is approximately 100 s, far below R1's 14 400 s threshold. R1 therefore fires on the vast majority of legitimate baseline packets. This is by design: R1 targets deployments where individual

BMSes receive firmware at most once per maintenance window (hours to days), not continuous streaming. The threat model (§2) assumes infrequent OTA updates; E7 demonstrates what happens when that assumption is violated, and its FP rate should be interpreted as a sensitivity bound, not an operational FP measurement.

Per-strategy adaptive mimicry (E21). E21 probes each rule's documented blind zone with five near-threshold attack strategies, each labelled ATTACK and emitted in its own campaign over 30 repetitions (1770 attack events total). Each strategy is emitted in isolation with a controller state reset between strategies so that the drop, if any, is attributable to the strategy's own target rule; a shared campaign instead lets one strategy's rule fire arm the per-source hold register and cascade drops onto later same-source strategies (§7.2), which inflates the apparent catch rate. The strategies are: (i) *fanout-sub* keeps fanout at exactly 4 per 60 s window across six disjoint BMS groups so R5 does not fire; (ii) *fanout-three* fans a rollout to three BMS, one below the R5 threshold; (iii) *R4 dead-zone* uses payload sizes inside the range-match dead zone so R4 does not fire; (iv) *combined* mixes sub-threshold fanout with the R4 dead zone; (v) *R1 late-replay* re-hits the same BMS with a second replay in each pair.

Table 8
E21 rule-fire breakdown per mimicry strategy under per-strategy isolation, summed over 30 campaigns (1770 attack events total). Each strategy is designed to evade a specific data-plane rule; "caught" counts events the controller dropped, "missed" counts everything else. Under isolation, only *R1-late* produces any drop; every sub-threshold strategy evades entirely. Missed events escape the data plane and would need control-plane or endpoint recovery.

Strategy	N	R1	R2	R4	R5	caught	missed
fanout-sub	720	0	0	0	0	0	720
fanout-three	540	0	0	0	0	0	540
R4-deadzone	90	0	0	0	0	0	90
combined	360	0	0	0	0	0	360
R1-late	60	30	0	0	0	30	30
Total	1770	30	0	0	0	30	1740

The per-strategy fire counts are in Table 8 and the per-strategy detection rates in Fig. 11. R5 fires on zero of the 720 fanout-sub and 540 fanout-three events, confirming the sub-threshold-fanout blind zone on the hardware; R4 fires on zero of the 90 dead-zone events, confirming the range-match blind zone. Under per-strategy isolation the only strategy with any detection is *R1-late*, where R1 fires on the second replay of each pair (30/60, 0.50); the *combined* strategy, which a shared continuous campaign had scored as dropped at a usable rate, is in isolation fully evasive (0/360), confirming that the earlier apparent catch was the per-source hold cascade rather than a target-rule fire. Across all strategies the data plane drops only 30 of 1770 attack events (1.7%); the cross-strategy F1 is 0.033 [0.022, 0.049]. The residual 1740 events pass the data plane without a drop. These are the rule set's blind zones under adversarial input design; endpoint signature verification and operator-side rollout monitoring remain needed to close them.

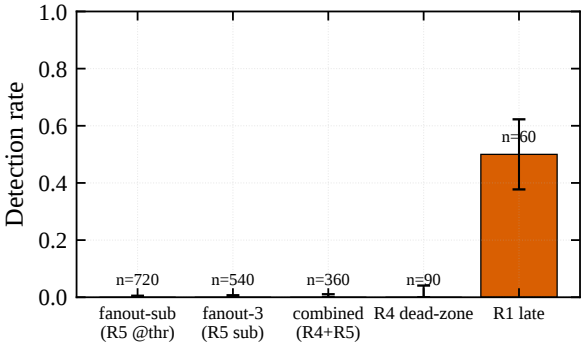


Figure 11: E21 per-strategy adaptive-mimicry detection rate over 30 campaigns; error bars are Wilson 95% CIs. Only `r1-late` produces any detection (R1 on the second replay, 0.50); every sub-threshold strategy evades entirely (0/N with Wilson upper bounds $<5\%$). Cross-strategy BCa $F_1 = 0.033$ [0.022, 0.049].

TCP-segmentation evasion battery (E23). The data plane parses the MQTT/OTA header per packet; it performs no TCP reassembly. We measure the consequence directly with a 5×5 battery: five segmentation evasions against the five header-dependent rules, 10 trials per cell (250 trials), each cell emitted after a controller reset so the outcome is attributable to the cell's own rule. Figure 12 reports the observed per-cell drop rate. The three *header-split* evasions (`split-mqtt-fh`, `split-topic`, `split-ota-hdr`) cut the single OTA-carrying PUBLISH across two segments so that no segment presents a parseable header; all three evade *every* rule, including the source-keyed R2 (0.00 across the row), because R2's evaluation is gated on the same per-packet OTA parse that the split defeats. The `out-of-order` evasion (halves sent tail-first) likewise evades every parse-dependent rule (R1/R4/R5/R6 = 0.00) while R2 survives (1.00), since its first-sent segment still begins with the MQTT control byte. Only `retransmit-dup`, which leaves an intact PUBLISH on the wire, yields parse-dependent detection, and only probabilistically (R1 0.40, R4 0.60, R6 0.80). No cell fires a rule the threat model did not predict, so no evasion induces a spurious drop. The R5 column is not a faithful fanout test (its single-PUBLISH base attack cannot reach the $\geq$ 4-BMS fanout threshold even un-evaded), so its entries are shown for completeness, not read as fanout-evasion results. The practical consequence: an adversary who fragments the OTA PUBLISH at the TCP layer defeats the header-dependent rules wholesale, so a stateful

upstream reassembly point (or a reassembling endpoint check) is required to close this class.

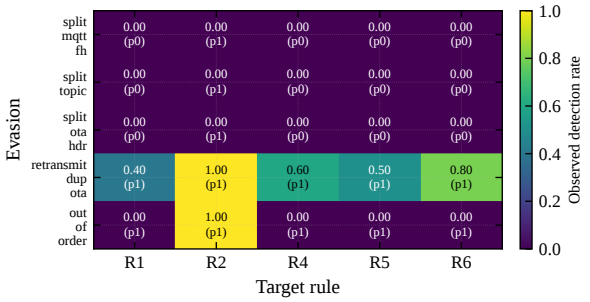


Figure 12: E23 TCP-segmentation evasion battery: observed per-cell drop rate over 10 trials/cell (250 trials), each cell reset-isolated; cell text is observed rate (predicted in parens). Header-split evasions defeat every rule including the source-keyed R2; only `retransmit-dup` (intact PUBLISH on the wire) yields parse-dependent detection, and only probabilistically. The R5 column does not exercise genuine fanout (single-PUBLISH base attack) and is shown for completeness.

Long-horizon slow-cadence trace (E7b). E7b runs the deployed gated build over a realistic deployment horizon: 200 events (100 benign updates plus 100 rollback attacks) over 7.9 h of wall clock, with per-BMS cadence resampled from 18–22 ks (median inter-update interval ≈20,000 s). Two effects show up. First, the slow cadence does not by itself silence the behavioral rules. 20 of the 100 benign updates reach the controller (the other 80 pass natively at the data plane with no decision); R1 fires on all 20 of them, where interleaved re-hits fall inside its window, and the arbiter demotes each to PASS through Gate A. R5 fires on 0 benign updates, since it is r2-gated (§6.4). No benign update is dropped (0 false positives across 100 updates; precision 1.000), consistent with the 0/450 of E12 (§6.3). Second, R6 catches all 100 of 100 rollback attacks (recall 1.000, F1 1.000). Unlike E19′, this trace has no first-contact misses: over the long horizon each BMS receives a benign update that establishes its version high-water mark before any rollback targets it, so R6 always has a baseline to compare against. E7b therefore complements E19′. Where E19′ isolates the cold-start gap with stochastic first-contact rollbacks (§6.4), E7b shows that once baselines are warm the deployed build sustains $P = R = F_1 = 1.000$ across an 8 h trace, with the arbiter containing every benign rule fire.

6.6. RQ4: CPU-based IDS baselines

Finding: off-the-shelf Suricata and Zeek fire on the fanout shape of legitimate rollouts and cannot suppress it, so they plateau at precision 0.67–0.70 on the same traffic where OTA-Shield reaches 1.00. OTA-Shield's edge is that R5 is gated in the data plane on unauthorized sources, so authorized rollouts never fire it, while the RAT arbiter demotes the R1 re-pushes that do fire. The gap is the policy-aware pipeline, not baseline expressiveness.

Conventional CPU-based IDS, Suricata and Zeek, run on the same port-mirrored traffic a programmable switch can tee, so they are the natural apples-to-apples baseline for an OTA detector that otherwise requires a Tofino 1 pipeline. RQ4 replays one E1 PCAP through five baseline configurations that together span the expressive ceiling reachable without a policy-aware arbiter.

CPU-based IDS baselines (E10). We captured a PCAP of a single E1 trial (90 events: 60 attack + 30 legitimate) and replayed it through four baseline configurations plus one apples-to-apples combination. R6 is omitted from this mapping because the E1 PCAP contains no version-rollback events; the R6 evaluation is reported separately in E19. Table 9 collects the six variants; the text below summarizes the arbitration gap they expose.

Table 9
E10 CPU-IDS baselines on the E1 PCAP replay (90 events: 60 attack + 30 legitimate). Variants (i)–(iv) are CPU-IDS configurations; variants (v) and (vi) post-process variants (i) and (iii)'s alerts respectively through the same RAT arbiter OTA-Shield uses in-band (the apples-to-apples controls).

Variant	Prec.	Recall	Acc.	TP	FP
(i) Suricata min-effort	N/A	0.000	0.333	0	0
(ii) Suricata stateful	0.691	0.783	0.622	47	21
(iii) Suricata permissive	0.667	1.000	0.667	60	30
(iv) Zeek domain-aware	0.698	1.000	0.711	60	26
(v) Suricata + RAT	N/A	0.000	0.333	0	0
(vi) Suricata permissive + RAT	1.000	0.500	0.667	30	0

Variant (i) maps R1, R2, R4, R5 to the closest Suricata 7.0.3 primitive (threshold, content byte match, dsize) and produces zero alerts; it is retained as the expressive-floor anchor. Variant (ii) uses Suricata's richer stateful primitives: `xbits(track ip_dst, expire 14400)` for per-destination replay memory, `flowint` accumulators for byte totals, `threshold by_src count 5, seconds 60` for R5 fanout, a negated allow-list for R2, and `flow:established` to avoid matching synthetic mid-flow packets. Variant (iii) is the same ruleset with `flow:established` relaxed to upper-bound what Suricata rule primitives can express on a synthetic PCAP; it catches every attack but flags every legitimate rollout that trips the same R5 fanout primitive. Variant (iv) replaces the rule DSL with a script-based Zeek detector: a per-destination `last_seen` table (14 400 s expiry) for R1, a per-source fanout set (60 s tumbling window) for R5, an OTAS magic parser for payload inspection, and an R2 authorized-source check. Variant (v) post-processes variant (i)'s Suricata alerts through the same RAT arbiter the OTA-Shield controller runs in-band, giving a rule-DSL detector access to RAT-based demotion. Because variant (i) emits zero alerts on E1, the arbiter receives zero inputs,

and the combined system yields recall = 0: giving a rule-DSL detector access to RAT-based demotion does not rescue the expressive floor when the rules themselves never fire. Variant (vi) closes that gap by arbitrating the alerts from variant (iii) (which actually fire) through the same RAT arbiter. Precision climbs to 1.000 (the RAT demotes every false-positive legitimate-rollout alert), but recall drops to 0.500: half the attack events share the authorized source IP and payload-size envelope of a legitimate rollout, and the RAT arbiter, operating purely on Suricata's 5-tuple + timestamp view, cannot distinguish a rapid-replay attack from a benign rollout on the same flow key without per-flow R1/R4 bits. This is exactly the signal OTA-Shield keeps in the data plane: the arbiter upgrades HOLD → PASS or HOLD → DROP using the rule set that fired on *this* flow, not a post-facto coincidence of 5-tuple and time window. Giving a CPU-IDS baseline the RAT as a black-box post-filter helps precision but silently drops half the attacks.

The pattern across variants (ii)–(iv) is consistent: once the baselines receive the long-horizon state and cardinality primitives needed to catch the attack events, they also flag every legitimate rollout that happens to ride the same primitive. Stateful Suricata and domain-aware Zeek both reach R = 0.78–1.00 but plateau at P = 0.67–0.70 because neither can tell an authorized rollout from a campaign. OTA-Shield avoids these false positives in two ways the baselines cannot: R5 is gated in the data plane on unauthorized sources, so an authorized fleet fanout never fires it, and the RAT arbiter demotes the R1 re-pushes that do fire. This is a policy-awareness gap, not an expressive gap: the detector primitives are reachable on CPU; the data-plane authorization gating and the rule-to-policy arbitration that suppress the legitimate-rollout fires are not. R6 remains inexpressible in Suricata's rule DSL in all variants; the Zeek variant approximates it with the OTAS parser but still cannot distinguish authorized rollbacks from unauthorized ones without RAT-equivalent policy state.

Variant (vii): CPU port of the OTA-Shield rule arbiter. To isolate the in-network performance advantage from the rule expressiveness advantage, we wrote a Python reference implementation of the exact R1–R6 logic and RAT-aware arbitration the data-plane binary executes. On the same 90-event E1 PCAP, the CPU port produces $P = R = F_1 = 1.00$ (TP=60, FP=0, TN=30, FN=0), matching the data-plane binary exactly. Sustained throughput on a single CPU core is $\sim 7 \times 10^5$ packets/s in unoptimized Python on three captures of 90, 4841, and 7310 packets respectively. Compared against the Tofino 1 binary's line-rate forwarding budget on a 25 GbE port ($\sim 3.7 \times 10^7$ pkt/s at 64 B), the CPU port is roughly 50× slower. The expressiveness gap from variants (i)–(vi) collapses to zero once the CPU detector implements the same arbitration we run in the data plane; the remaining gap is performance and resource cost, not detection quality. This separation of concerns motivates §6.7 (latency, throughput, capacity, portability), where the in-network deployment is justified on cost rather than on detection ceiling.

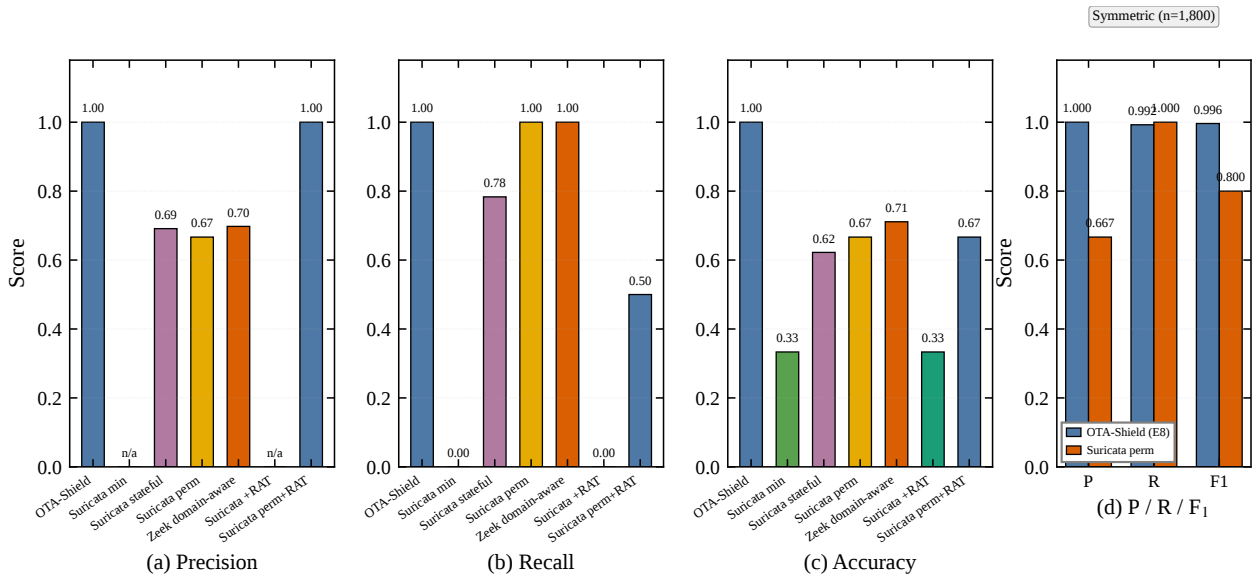
Symmetric comparison on the full E8 population. The E10 comparison above runs each baseline once on the single-trial E1 PCAP (90 events), whereas the headline OTA-Shield result (E8) uses 20 stochastic trials totalling 1800 events. To match sample size, we reconstructed a per-trial PCAP from each E8 stochastic trial (same packet format: scapy PA-only, MQTT/OTAS headers from the recorded event parameters) and ran Suricata stateful-permissive (the best-recall variant (iii)) across all 20 trials. The pooled result on 1800 events (1200 TP, 0 TN, 600 FP, 0 FN) is P = 0.667, R = 1.000, F_1 = 0.800, identical to the single-trial E1 number, confirming that the asymmetry was in sample size, not in the methodological conclusion. The stochastic ordering and ephemeral port variation in the E8 trials do not alter Suricata's decision for any event: variant (iii)'s rules fire on every attack event and on every legitimate rollout regardless of inter-arrival order, because neither R1's `xbits` expiry nor R5's per-source threshold distinguishes ordering within the 60-second window. Compared with OTA-Shield's E8 result (P = 1.000, R = 0.992, F_1 = 0.996), the precision gap is $\Delta P = 1.000 - 0.667 = 0.333$ and the recall gap is 0. The 600 false positives are 600 legitimate rollout events that Suricata's stateful rules flag on their fanout shape and cannot demote, having no notion of an authorized rollout. OTA-Shield does not flag them at all, because R5 is r2-gated (§6.4). On this workload the precision advantage is therefore a *data-plane* property, the r2-gating of R5, not a control-plane demotion. The arbiter's complementary role shows up on the per-BMS replay cohort: where a legitimate rollout instead trips R1 (a re-push inside its window), Gate A is what admits it, as E12 demonstrates directly.

6.7. RQ5: Latency, throughput, capacity, and portability cost

Finding: median latency is 34.0 ms (p95 46.6 ms), the override table holds under sustained install pressure, and 4 of 8 parser variants miss on a single-MQTT-profile sweep. Costs land well inside the operational envelope.

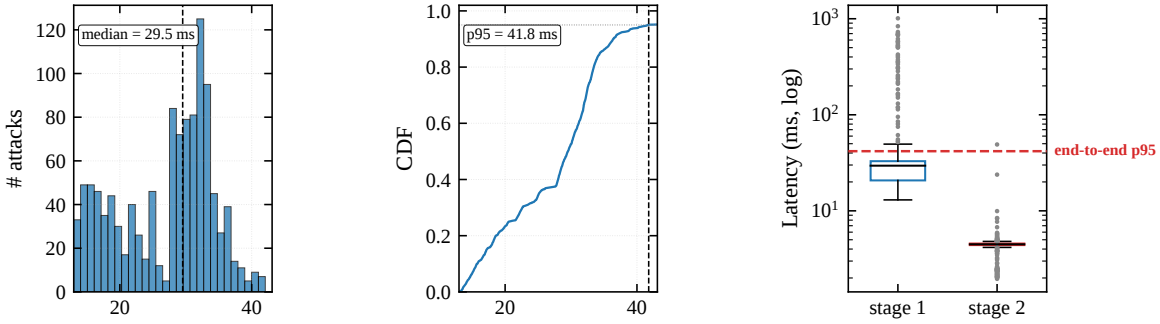
RQ5 reports the cost side of the ledger: end-to-end and per-stage detection latency (E14 on the E8 dataset), controller-side digest ingestion rate (E11), session-override table capacity (E13), and the parser's sensitivity to MQTT-profile variations (E18).

Detection latency and per-stage breakdown (E14). Using the timestamps already written to the E8 logs we decompose end-to-end latency into two measurable stages on the 1200 attack events reported in Fig. 14.



(a) Precision (b) Recall (c) Accuracy (d) P / R / F_1

Figure 13: CPU-based IDS baselines vs. OTA-Shield. **Panels (a)–(c): asymmetric comparison on the E1 PCAP replay** (90 events: 60 attack + 30 legitimate), seven systems, three metrics. The minimum-effort Suricata mapping and its RAT-arbitrated combination (variant v, apples-to-apples) both floor at recall = 0; stateful Suricata and Zeek domain-aware recover recall to 0.78–1.00 but plateau at precision = 0.67–0.70 because they fire on the shape of legitimate rollouts and cannot suppress it. Variant (vi) = permissive Suricata + RAT reaches precision = 1.00 but its recall drops to 0.500 because RAT without per-flow R1/R4 bits cannot separate same-source rapid-replay attacks from legitimate rollouts. OTA-Shield reaches P = R = 1.00 on the same PCAP. The gap is policy-awareness, not the expressive power of any single primitive: OTA-Shield gates R5 on unauthorized sources in the data plane and demotes R1 re-pushes at the arbiter. **Panel (d): symmetric comparison on the full E8 population** (1800 events across 20 stochastic trials): Suricata stateful-permissive achieves P = 0.667, R = 1.000, F_1 = 0.800; OTA-Shield achieves P = 1.000, R = 0.992, F_1 = 0.996. Precision gap $\Delta P = 0.333$; recall gap = 0.



(a) End-to-end latency (ms) (b) End-to-end latency (ms) (c) Per-stage breakdown

Figure 14: End-to-end detection latency on 1200 attack events from the stochastic experiment: (a) histogram, (b) empirical CDF, (c) per-stage box plot on log scale. Stage 1 = send → digest arrival (generator + Tofino 1 pipeline + gRPC transport); stage 2 = digest arrival → controller decision (Python arbitration); end-to-end = stage 1 + stage 2. Medians 29.5/4.5/34.0 ms; p95 values 41.8/4.8/46.6 ms. Latency is restricted to attack events because the benign workload interposes inter-trial state-reset barriers (∼2.7 s) between trials; those barriers are driver-side synchronization, not per-event detection latency, so pooling them would overstate the on-path tail.

Stage 1 (send time → controller-side digest arrival) captures the packet's flight from the generator plus the Tofino 1 pipeline plus the gRPC digest transport. Stage 2 (digest arrival → controller decision) captures the RAT arbitration path on the Python side. Medians are stage 1 = 29.5 ms and stage 2 = 4.5 ms, and the additive end-to-end median is therefore 34.0 ms = 29.5 ms + 4.5 ms (within the 0.1 ms rounding the macro carries). At p95 the two stages contribute 41.8 ms and 4.8 ms for an end-to-end p95 of 46.6 ms (= 41.8 + 4.8 within rounding). The per-stage breakdown makes explicit that *"in-switch"* does not mean *"in-switch latency"*: the dominant term is stage 1 (generator emission + Tofino 1 pipeline + gRPC transport), and the ASIC pipeline itself is a microsecond-scale component inside that ≈29 ms envelope. Stage 2 contributes a small but non-negligible 4.5 ms tail to every decision because the controller's RAT-arbitration path is Python and runs

synchronously on digest receipt; stage 2 is the largest software-side optimization target if a tighter SLA is required. The latency distribution has a short head and a small tail. The tail is driven by two infrequent controller-side events: a state-reset barrier between trials, during which digest processing pauses briefly; and a timer-based override-TTL sweep that contends with the main digest loop for the gRPC channel when many session overrides expire at once. Neither event is on the path of a data-plane fire-to-detection decision; both affect when the controller's Python loop returns to the digest receive call after the barrier. For an operational SLA measured in seconds (OTA campaigns run over minutes to hours), this tail is tolerable; for a tighter SLA, moving timer-driven override expiry out of the main thread would flatten it.

Controller digest-channel sustainable rate (E11). E11 measures the controller's classify/MQTT-parse digest ingestion path (phases 1 and 2), not the phase-6 HOLD path: the generator sends MQTT PUBLISHes to a destination outside the authorized BMS fleet with a non-OTA topic prefix, so no rule fires and no override installs. The pipeline still emits a phase-1 or phase-2 digest per parsed packet, and it is this digest delivery the controller has to sustain. Sweeping 100/500/1000/2000 pps, the controller observed 35967/36000 digests for an aggregate loss of 0.09% (worst per-rate 0.20% at 100 pps, statistical noise on the smallest window). The scapy generator caps offered load near 2,200 pps, so this is a lower bound on the controller's classify-digest rate, not an upper bound on switch capability or on the HOLD-digest path. The HOLD-digest path is exercised separately at the override-install rate measured in E13: every R5 fire emits one phase-6 digest, the controller arbitrates, and installs at most one override per source. "No rule fires, no phase-6 digest, but a classify/MQTT-parse digest still streams" is the precise mapping between E11's offered traffic and the measured digest count, and resolves the apparent tension with the "no rule fires ⇒ no digest" phrasing used elsewhere, which referred specifically to phase-6 HOLD digests. Line-rate testing of either path requires a DPDK generator such as P4TG [44, 45] and is deferred to future work.

Override-table capacity (E13/T1.7). The deployed `session_action_override` table keys on the full 5-tuple (`src_addr`, `dst_addr`, `dst_port`, `protocol`, `src_port`) and is sized at 256 entries; we confirmed this key shape and size by `bfrt_grpc` inspection of the running pipeline on every trial of T1.7. The 5-tuple key is a deliberate isolation property: an `ALLOW` installed for one flow does not authorize unrelated traffic from the same source, since an alias 5-tuple with different ports cannot collision-match the entry. T1.7 measures the deployed table's capacity: 200 concurrent sources, one `ALLOW` each, install cleanly across 20 trials (4000 inserts in total) with 0 `RESOURCE_EXHAUSTED` rejects (Clopper–Pearson 95% upper bound 13.91%), so the 256-entry table has headroom for a 200-source fleet at one HOLD per source. The capacity that matters under this key is *5-tuple* cardinality, not source-IP cardinality: a single source fanning out across many BMSes with churning ephemeral ports can generate many distinct 5-tuples, which is why the table is paired with a 5 s fail-closed TTL and why a source-IP aggregation variant is retained for fanout-heavy deployments. The remainder of this paragraph reports that variant.

Source-IP aggregation variant. The pilot 5-tuple-keyed override table at 1024 entries saturates within seconds under sustained R5 fanout: a 1000 pps burst against 50 BMSes with unique ephemeral source ports floods the table before the 5 s TTL can evict stale entries. The aggregation variant instead keys on source IP only, so every HOLD from one source collapses into a single table entry regardless of how many BMSes the fanout covers. One override per source covers that source's entire fanout until TTL expiry.

We run the install-pressure workload (sweep offered rate {100, 250, 500, 1000} pps for 10 s, realized rate capped near 500 pps by the scapy raw-socket ceiling) from a single authorized source. The E13 realized-rate ceiling (≈500 pps) is lower than the E11 digest-channel ceiling (≈2,200 pps) because the two workloads stress different per-packet scapy costs: E11 sends a uniform PUBLISH stream to a single unauthorized destination with no rule firing and no override install, while E13 sweeps distinct source IPs and triggers a fresh R5 HOLD per source that the generator must follow up with an install-ack bookkeeping step. The per-frame encoding cost is therefore higher in E13, and the same scapy raw-socket generator delivers a lower realised rate for the same offered load. This is a property of the traffic shape and the userspace generator, not a different hardware bound on the switch. Under the aggregation variant, peak active override entries stay at 1 across all four windows, despite 2051 distinct 5-tuples in the largest window. To measure the variant's cap on source-IP cardinality, we sweep {1, 10, 32, 64, 100, 200} distinct source IPs at 500 pps for 10 s each (Fig. 15). Peak active override entries track source-IP cardinality exactly (1, 10, 32, 64, 100, 200), and the switch records 0 RESOURCE_EXHAUSTED rejects across 12476 controller decisions. Table occupancy is therefore bounded by source-IP cardinality, which typically is 1–2 in a BESS fleet regardless of 5-tuple churn from ephemeral ports. We retain this source-IP aggregation as a compile-time knob for fanout-heavy deployments; the *default in the deployed pipeline is the 5-tuple-keyed table* characterised by T1.7 above, chosen for its per-flow authorization isolation rather than for the lower occupancy of the aggregation variant.

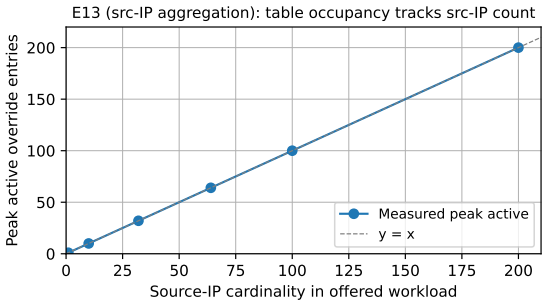


Figure 15: E13 (source-IP aggregation variant). Peak active override-table entries versus offered source-IP cardinality at 500 pps per window, averaged over a 5 s TTL. Measurements fall exactly on $y = x$: one table entry per distinct source IP, independent of 5-tuple fanout. Zero installs are rejected up to 200 source IPs in our deployment pipeline.

Scope of the reference channel (E18). E18 sweeps MQTT-level parameter variations (topic length, QoS level, retain flag) to characterize how tightly the parser is bound to the reference-channel assumptions. This is a scoping result rather than a portability claim: it does not show that the construction ports to an arbitrary MQTT profile, only that deviations from the reference framing require per-channel re-engineering of the parser. Table 10 reports the observed pipeline outcome per variant in the core MQTT-parameter sweep.

Table 10
E18 reference-channel portability across the 8-variant MQTT-parameter sweep on the deployed parser. PARSED = the parser extracted a valid OTA header; PARSER-MISS = the packet was classified as non-OTA and forwarded unflagged. No variant is dropped; the determining factor is the 32-byte topic slot, not QoS or the retain flag.

Variant	Parameters	Outcome
baseline	topic=32 B, QoS=1, retain=0	PARSED
QoS=2	topic=32 B, QoS=2, retain=0	PARSED
retain=1	topic=32 B, QoS=1, retain=1	PARSED
QoS=0	topic=32 B, QoS=0, retain=0	PARSED
short topic	topic=16 B, QoS=1, retain=0	PARSER-MISS
long topic	topic=64 B, QoS=1, retain=0	PARSER-MISS
QoS=0, short	topic=16 B, QoS=0, retain=0	PARSER-MISS
QoS=0, long	topic=64 B, QoS=0, retain=0	PARSER-MISS

The deployed parser parses every variant that keeps the 32-byte topic slot (the reference channel and its QoS-2, retain, and QoS-0 variants) and misses every variant that changes the topic length, which shifts the OTA-header offset past the fixed slot. No variant is dropped, and QoS and the retain flag do not affect parsing. This confirms that the OTA channel used in the evaluation is a reference design rather than a universal model of BESS OTA practice: an operational deployment on a different MQTT profile would require a variable-length topic parser, which would add MAU stages. This is consistent with the scope framing in §1. Under the full 8-variant sweep, the deployed binary parses 4/8 variants end-to-end and misses 4 (all misses driven by topic-length errors); agreement between the predicted and observed per-variant outcome is 7/8. The single disagreement is the QoS=0 variant at the reference 32-byte topic, which parses observably even though the missing 2-byte packet identifier was expected to misalign the OTA header: the deployed parser is more tolerant of the QoS=0 offset shift than the byte layout suggests. The practical implication is that the QoS=0 parser branch prepared for recompile targets the variable-topic-length misses, which are topic-length errors rather than QoS-offset errors. This does not affect the headline portability claim (4/8 parse under the current binary).

200-BMS fleet replay (E1′, E8′). The reference numbers reported in §6.3–6.6 are bounded by a 50-BMS testbed. To characterize the design at deployment-scale fleet size, we replay two fleet-scale traces against the live controller. E1′ applies a superposed-Poisson workload with 200 synthetic BMS endpoints emitting 4841 events at an aggregate rate of 40 Hz. E8′ scales the stochastic E8 mix to the same fleet size, producing 7310 events. Per-BMS rate, payload size, and version-bump probability are inherited from the 50-host long-baseline measurement; only fleet size and the synthetic addresses 10.0.2.60–10.0.2.209 are extrapolated. Before the run, the controller is restarted with the extended manifest so the override table starts empty.

Table 11
50-BMS vs. 200-BMS precision and F_1. Recall is preserved at 1.000 for both fleet sizes because detection coverage scales linearly with fleet size (the R1/R4/R6 rule chain is intact). The precision collapse at 200-BMS is attributed to the 64-entry controller/manifest BMS cap. The data-plane `bms_ip_to_idx` table is itself compiled at 128 entries (Table 16), so legitimate packets addressed to BMS 65–200 are unmapped and take the fail-closed DROP path.

Experiment	Fleet	Precision	Recall	F_1
E8 (50-BMS headline)	50	1.000	0.992	0.996
E1′ (200-BMS, det.)	200	0.195	1.000	0.327
E8′ (200-BMS, stoch.)	200	0.292	1.000	0.453

The replay exposes two findings the 50-BMS workload cannot. First, recall is preserved: across both scenarios no rollback attempt is missed (0 and 0 attack-side false negatives), so detection coverage scales linearly with fleet size and the R1/R4/R6 rule chain remains intact under sustained 40 Hz fanout. Second, the controller caps the active BMS-index manifest at 64 entries; the data-plane `bms_ip_to_idx` table itself is compiled at 128 entries (Table 16), so the bound here is operational, not a hardware table-size limit. At 200 BMS the manifest cap leaves BMS beyond the first 64 unmapped: a legitimate packet addressed to one of them misses the index table, is signalled `action_code=2` by the data plane, and lands as a `terminal_fire` DROP at the arbiter. The aggregate result is 0.195 precision and 0.327 F1 on E1′ (917 TP,

45 TN, 3781 FP, 0 FN), and 0.292 precision and 0.453 F1 on E8′. Digest loss at fleet scale is modest (2.0% on E1′, 4.0% on E8′), so the digest channel itself is not the bottleneck.

The construction therefore holds at 200-BMS scale on the rule side but not on the table-population side. The 64-entry cap is a deployment-time scope rather than an algorithmic limitation: raising it to the 128 the data-plane table already provides is a controller/manifest change, and covering a fleet beyond 128 additionally requires a P4 recompile to widen the BMS-index, R1, and override tables, after which the same controller and manifest cover the full fleet. We report E1′ and E8′ alongside the 50-BMS numbers to make explicit which result scales with fleet size (recall) and which depends on a recompile (precision at fleet scale).

7. Discussion and Limitations

7.1. Interpretation of results

On the deployed gated build, E4 shows the terminal rules (R2, R4, R6) carry the attack precision without the arbiter: disabling the RAT leaves precision at 1.0, because R5 is gated on `r2_fired` and never fires on an authorized rollout. The arbiter's value is on rollback admission (E19), not an R5 precision delta. The boundary sweeps (E9) map hardware-imposed limits (R4's 64 KiB dead zone between 2.0–2.0625 MiB and R1's 18.2-hour wrap-around) into the threat model rather than hiding them. No single rule is evasion-proof; the multi-rule architecture is defense-in-depth within a reference channel. The remainder of this section states the limitations we measured, and for each names a realistic next step.

7.2. Limitations

The three deployment paths this scope implies were set out early in Fig. 1 (§2): a supported cleartext segment after TLS/VPN termination, and two paths the evaluated build does not cover, a brokered path (E20) and an encrypted or TCP-segmented path (E23). We group the measured limitations into placement, parser and protocol, adversary-adaptation, scaling and capacity, and artifact and reproducibility limits, and for each name a realistic next step.

Placement limits: TLS/VPN scope. OTA-Shield requires placement after TLS/VPN termination or access to plaintext OTA metadata. The R1, R4, R5, and R6 rules all read fields that the parser extracts from the cleartext MQTT PUBLISH and the 20-byte OTA header; on a flow still inside a TLS or VPN tunnel at the switch, the parser cannot recover any of those fields and none of the rules can fire. The deployment requirement is therefore that an upstream broker, IIoT gateway, or VPN concentrator terminate the encrypted leg, and that OTA-Shield sit on the cleartext segment between the terminator and the BMS fleet. Next step: a cross-protocol side-effect rule (R7) correlating post-flash bursts in NTP, DNS, and DHCP, which would cover encrypted OTA channels whose payload cannot be parsed.

Artifact and reproducibility limits: comparative-evaluation scope. Our baseline comparison spans two tiers run on our own traffic, off-the-shelf signature IDS (Suricata, Zeek) and a CPU port of OTA-Shield's own rule and arbiter logic (E10), plus a cited-numbers table positioning OTA-Shield against prior in-network and ICS detectors. We did not run a prior ICS-specific IDS on our own PCAPs. The closest neighbour, a P4-MQTT topic-enforcement detector [16], targets broker ACL violations rather than OTA fleet coordination and runs on a software (BMv2) target, so a like-for-like port onto the E1/E8 captures would convert that prose positioning into a measured point; we leave that port to future work. The hardware experiments require an Intel Tofino 1 with BF-SDE 9.13.2 and the emulated BMS testbed: the released artifact ships the P4 source, controller, and result aggregators, but the multi-hour raw run logs and the live testbed are not redistributable, so hardware-bound results are inspectable through the aggregation path rather than re-runnable from the public repository.

Parser and protocol limits. The parser is bound to the reference MQTT framing measured in E18: of the eight QoS and topic-length variants exercised, 4 parse end-to-end and the remainder miss on the fixed 32-byte topic-slot assumption. TCP segmentation (E23, §6.5) splits the OTA header across segments so that no single packet carries the parseable fields, and the data plane performs no reassembly, so a header-split attack evades all rules. Variable-length topic parsing and a reassembly shim are the corresponding extensions.

Placement limits: brokered-MQTT source-IP collapse (E20). Where the OTA path traverses an MQTT broker rather than reaching the fleet directly, a topology some grid profiles mandate, every PUBLISH on the BMS-facing wire carries the broker's source IP. We emulate this on the testbed (80 trials × 30 publishes across one benign and three attack scenarios, with post-relay packets carrying the broker source IP) and the controller fires R2 (unauthorized source) on all 600 of every scenario's events, benign and malicious alike, dropping the benign broker-relayed rollout in full (600/600 false positives). The source-IP-keyed rules cannot separate authorized from rogue publishers once the broker collapses their addresses, so an IP-only authorization table yields no precision in this topology (negative control: precision collapses, F1 undefined under all-drop). The mitigation is to key authorization on a publisher identity carried end-to-end through the broker rather than on the source IP; the data-plane header reserves a 64-bit hash-hint field for exactly this

purpose, but the controller-side publisher-id keying is not implemented in the evaluated build. Brokered deployment is therefore a documented gap: we report no publisher-id F1, because with no publisher-id logic in the controller any such number would reflect the IP-keyed all-drop behavior rather than a working publisher-id RAT.

Adversary-adaptation limits: RAT trust surface. Detection accuracy rests on a correct RAT manifest. An attacker with write access before controller start, or a privileged insider authoring entries, can widen a rollback window and suppress R6 for that scope. Because Gate B matches on (*src*, *dst*, *size*, *version*) rather than firmware content, a compromised authorized source can also inject an image whose declared version falls inside a legitimate window, and a sub-threshold fanout (≤ 4, which E21 shows fires R5 on 0/1260 events) carrying such a version passes both gates without ever raising a HOLD. Against an adaptive adversary who reshapes cadence and volume, held-out mimicry coverage falls to 1.7% of attack events (E21). The mitigations are operational: short-lived per-rollout entries, a signed manifest (the controller verifies an ed25519 signature at load and hot-reload and retains the last-known-good cache on failure, E12b Phase B), bounded `rollback_window` spans, and a WARNING-level audit record on every Gate B demotion. Signature verification does not catch replay of an older valid manifest; binding manifests into a monotonic chain is left to operator integration.

Scaling and capacity limits: per-source HOLD gate. The `hold_armed_reg` gate is indexed by the low 8 bits of a CRC16 of the source IP (256 buckets). On the deployed build it arms only on R5, and because R5's counter is gated on `r2_fired` behind the terminal R2 drop, the gate does not arm on any reachable input and is inert; this is why E12 and E19' report zero benign false positives. (An earlier build armed the gate on any rule fire, which is the configuration recorded in Appendix C.) The 256-bucket index remains a latent limitation for any future build that re-arms the gate on unauthorized fanout: at 10 to 50 concurrent sources, birthday collisions would make it unreliable as a per-source discriminator, and widening the index to a 32-bit CRC32 over the 5-tuple is the mitigation. Rollback detection (R6) further assumes a prior reference version for the targeted BMS: an attack to a device with no baseline yields a no-decision, so lenient recall is 0.970 over devices with an established baseline and strict recall is 0.857 counting first-contact no-decisions as misses (§6.4, Appendix B).

Scaling and capacity limits: Bloom sizing for fleet fanout. The R5 fanout detector indexes each destination into three 1024-entry Bloom filters. Because it gates on `r2_fired==1` (§5), the load that matters is unauthorized fanout per 60-s window, not fleet size, and the deployed sizing is collision-free for the fleet sizes evaluated here. Sustained unauthorized fanout above that band would need wider registers; a 4096-entry resize fits in SRAM without an added MAU stage.
The remaining limitations, including the session-hash collision bound (E5′), the cadence-gated status of R1 (E7), and the override-table capacity evidence (E13), are summarised in Table 12.

7.3. Positioning relative to prior systems

No prior system addresses the same combination of BESS OTA semantics, in-pipeline data-plane enforcement, and policy-aware control-plane arbitration, so numeric comparison carries caveats. Table 13 places OTA-Shield alongside the closest prior work by detection domain, platform, and headline metric. Each row reports a number from the cited paper on that paper's own dataset; they are not numbers from a common benchmark, and no single row is directly comparable to ours.

On the same PCAP we benchmarked four CPU-IDS configurations (§6.6): a minimum-effort Suricata mapping (recall 0.000), a stateful Suricata mapping using `xbits/flowint/threshold` ($P = 0.691$, $R = 0.783$), the same mapping with `flow:established` relaxed as an expressive upper bound ($P = 0.667$, $R = 1.000$), and a Zeek domain-aware script-based detector ($P = 0.698$, $R = 1.000$). Variants (ii)–(iv) all cluster at $P \approx 0.67$–0.70 once they reach useful recall. The bottleneck on a CPU IDS is not data-plane line rate or detector expressiveness; it is that none of the CPU baselines can tell an authorized rollout from a coordinated campaign. OTA-Shield gets this from two policy-aware mechanisms the baselines lack: R5's fanout counter is gated in the data plane on the unauthorized-source flag, so authorized rollouts never trip it, and the RAT arbiter demotes the legitimate re-pushes that do fire R1. Its precision advantage at matched recall is therefore a property of the policy-aware pipeline as a whole, chiefly the data-plane r2-gating of R5 on this fanout workload, rather than of the control-plane arbiter alone.

7.4. Operational implications

For BESS operators, OTA-Shield provides a detection layer that operates at the network aggregation point (between the OTA server and the BMS fleet), where coordinated campaigns are visible but endpoint protections are blind. The RAT enables scheduled rollouts to proceed without false alarms, while the 5-second fail-closed override window limits the exposure of any single detection miss. The compile-time threshold constants require a P4 recompile to change (minutes, not hours), which is consistent with planned maintenance cycles but not with real-time threshold adaptation.

Table 12
Limitations of OTA-Shield outside the RAT trust surface. Each row names a limitation, its blind zone or failure mode, and an available mitigation path.

Limitation	Blind zone / failure mode	Mitigation
Simulation conventions	Parser assumes 32-byte null-padded topics and QoS = 1 with packet identifier; 4 of 8 swept variants miss the OTA header (E18).	Recirculation or a variable-length extraction pipeline; deployment-specific parser re-engineering per channel.
Switch line-rate throughput	Measured ceiling is ≈2,200 pps on a scapy generator; Tofino 1 line-rate is not exercised. Headline F_1 is bound to the generator, not to the ASIC.	DPDK-class traffic generator (P4TG) on dedicated ports, deferred to future hardware work.
Testbed fidelity	50 BMS endpoints share a single kernel, NIC, and clock; device-side jitter, clock skew, and per-device TCP behavior are not reproduced.	Evaluation on physical-device fleets; timing distributions not measured here.
Session hash collisions	17-bit CRC32 index has birthday-bound collisions $\approx N(N-1)/2^{17}$; R4 can false-positive or false-negative under concurrent flows. E5′ confirms the analytical bound at $N{=}10\,000$ with 35 observed collisions (1.40%, vs. predicted 1.88%).	Wider index (20-bit, 1 M entries) or set-associative table, at the cost of register memory.
R1 cadence gating	R1 applies only when per-BMS cadence is slower than $\tau_{\mathrm{R1}} = 14\,400$ s; under faster cadences R1 fires on legitimate traffic (E7) and must be disabled.	R2, R4, R5, R6 carry detection under faster cadences; control-plane replay detection for streaming cadences.
Threshold robustness	R5, R1, R4 thresholds are compile-time constants; no runtime adaptive thresholds.	By design: compile-time constants keep thresholds auditable and immutable; per-deployment recompile using E7 percentiles and E9 sensitivity.
Parser short-packet DoS	PUBLISHes with payload <20 bytes to OTA topics trigger a parser error and drop: fail-closed for attacks, but also drops legitimate short status messages.	Topic-prefix ACL upstream of the parser to scope fail-closed behavior.
Override table capacity	`session_action_override` keys on the full 5-tuple (compile size 256); occupancy bounded by 5-tuple cardinality under ephemeral-port churn (T1.7: 4000 inserts across 200 sources, 0 RESOURCE_EXHAUSTED).	Compile-time size is the knob; a source-IP aggregation variant (occupancy bounded by source-IP cardinality, E13) is preserved for fanout-heavy deployments.

Table 13
Position of OTA-Shield relative to related in-network detectors and CPU-based IDS. Each row reports a metric from the cited paper on *that paper's own dataset*; the dataset column makes explicit that these are not measured on a shared benchmark, and the numbers are not comparable across rows. The OTA-Shield row and the E10 CPU-IDS rows are the only rows measured on the same traffic as this paper.

System	Platform	Domain	Result on cited dataset (not comparable across rows)
Raj & Jin [14]	Tofino	Dynamic-data ML IDS	CICIDS: high accuracy
Chen et al. [15]	Tofino	P4 NIDS (distilled trees)	authors' trace: low-SRAM pipeline
AlSabeh et al. [27]	P4 sw.	DDoS entropy DPI	authors' DDoS trace: in-paper metric
Jaqen [12]	Tofino	Volumetric DDoS	authors' testbed: 380 Gb/s mitigation
MQTT pipeline [16]	BMv2 (soft)	MQTT enforcement	authors' emulator: 99.8%
Measured on the E10 PCAP in this paper:			
Suricata 7 (stateful) [8]	CPU	Signature IDS + xbits/flowint	E10 PCAP: P=0.69, R=0.78
Zeek (domain-aware)	CPU	Script-based IDS	E10 PCAP: P=0.70, R=1.00
OTA-Shield (this work)	Tofino 1	BESS OTA fleet attacks	E8 PCAP: F_1 0.996 [0.993, 0.999]

Table 14
Defence-in-depth coverage. Each layer catches a different fraction of the attack space; OTA-Shield is one of the three, not a replacement for the others.

Layer	What it catches
Endpoint signing (SUIT/Uptane)	Unsigned or mis-signed firmware at the BMS, regardless of how it arrives. Blind to *signed-but-malicious* firmware (stolen signing key) and blind to in-transit fanout.
OTA-Shield data plane	Naive coordinated campaigns (A1–A4 at their designed thresholds, plus A5 rollback via R6 and Gate B) at the network aggregation point. Blind to shaped adversaries that stay inside rule blind zones (E21); requires placement after TLS/VPN termination (§7).
Operator rollout dashboard	Slow-drift campaigns visible over hours to days through fleet-version skew. Too slow to stop a minutes-scale attack in flight.

Table 15
Resource footprint of OTA-Shield on Tofino 1, with rough comparison to a CPU-based alternative on the same traffic. Dollar-per-BMS-year assumes a five-year amortization of a USD 20 k switch over a 50-unit fleet.

Cost dimension	OTA-Shield (Tofino 1)	Suricata (Xeon Gold)
MAU stages used	12 of 12 (Table 16)	n/a
SRAM for rule state	3 KB (R5 Bloom) + 0.5 KB (R1)	host RAM
Session table (fleet-wide)	512 KB (two 64 K × 32 bit)	host RAM
Power per Gb/s (amortized)	∼70 mW/Gb/s	∼5 W/Gb/s
Throughput headroom	measured 2,200 pps (generator-bound); ASIC 25 Gb/s link, unmeasured	40–60 Gb/s [7]
Hardware cost (list)	$20 k (32×100 GbE)	$8–12 k server
$/BMS-year (50-unit fleet)	≈$80	≈$40

Table 16
Per-component data-plane resource placement, extracted from the deployed build's compiler output (the `context.json`/BFA of the binary loaded by `bf_switchd`). The pipeline occupies all 12 ingress MAU stages. "Largest match table" gives the dominant logical table per component and its entry count; "Stateful registers" lists the SALU-backed memory. Exact chip-relative SRAM/TCAM percentages require the compiler's MAU resource log, which was not retained for the deployed binary; we therefore report the authoritative allocation counts.

Component	Stage(s)	Largest match table (entries)	Stateful registers (entries)
Parsing + topic match	0	`topic_prefix` (8)	—
Session manager	0–1	`session_id` (64 K)	`session_bytes` (64 K), `first_ts` (64 K), `coarse_time` (1)
R2 unauthorized source	1	`r2_authorized_sources` (16)	—
R5 fleet fanout	1–6	`fleet_monitor` (64 K)	`bf_r0/r1/r2` (3×1024), `r5_count` (1)
BMS index map	5	`bms_ip_to_idx` (128)	—
R4 oversize	5–6	`r4_threshold` (range)	shares `session_bytes`
R1 replay	6–7	`secondary_rules` (256)	`r1_last_seen` (256)
R6 rollback	6	`rule_r6_rollback` (256)	`r6_bms_max_version` (256)
Policy engine	7–8	`set_src_idx` (256)	—
Override + hold cascade	9–10	`session_action_override` (256, 5-tuple)	`hold_armed` (256)
Deparser digests	11	6× learn-quantum digests	—

Whole pipeline: 12 of 12 ingress stages; 76 PHV containers (23×8 b, 25×16 b, 28×32 b); 14 SALU units; 10 TCAM blocks; 10 map RAMs.

7.5. Composing OTA-Shield with endpoint and operator-side defenses

OTA-Shield is one layer in a defense-in-depth stack, not a standalone detector. Table 14 sketches what each layer catches and misses.

7.6. Resource and deployment cost

Unlike CPU-based IDS, the data-plane layer pays its cost in MAU stages, SRAM, and amortized switch hardware rather than in cores and watts. Table 15 collects the measured resource footprint.

Table 16 gives the per-component placement extracted from the deployed build's compiler output. OTA-Shield uses all 12 ingress MAU stages; the long pole is the chain of dependent rule state (session bytes → R4/R5 thresholds → R1/R6 history → policy → override), each link of which must follow its producer's stage. The three range-match threshold checks (R1, R4, R5) account for the TCAM use, and the stateful

registers account for the SALU use, notably the three 1024-entry R5 Bloom filters and the two 64 K-entry session arrays. The full stage occupancy is the binding constraint on adding rules: a seventh behavioral rule with its own stage-dependent state would not fit without folding existing logic, which is one reason new detection logic is added in the controller rather than the data plane. (We also note that `bms_ip_to_idx` is sized at 128 entries in the compiled table; the 64-BMS figure quoted elsewhere is a controller/manifest-imposed cap, not the data-plane table size.)

Two caveats on the cost table. The switch cost is amortized over a single 50-unit fleet; in a larger substation with multiple fleets sharing the switch, the per-BMS figure falls proportionally. The power comparison is approximate: Tofino 1 ASIC power is roughly constant (the pipeline cost is marginal once the ASIC is on), while CPU IDS power scales with offered load.

At the 50-unit fleet measured here, Suricata is roughly 2× cheaper in $/BMS-year at list prices; we report Table 15 as a direct measurement at our evaluated scale rather than as a claim that OTA-Shield is cheaper per BMS in absolute terms. The in-network deployment contributes performance headroom and co-location with the forwarding path, not cost or unique detection capability at this fleet size. Two regimes where the tradeoff may favor in-network placement are: (a) multi-fleet substations where the switch is amortized across hundreds of BMS (e.g., a 200 + BMS substation, lowering the per-BMS cost to ≈$20); and (b) high-OTA-traffic loads where a CPU-bound IDS would need per-packet reassembly at rates the ASIC handles with zero incremental cost. Serving such a fleet means raising the 64-entry BMS manifest cap (a controller change up to the 128 the table already holds) and, beyond 128 BMS, a per-deployment P4 recompile to widen the index; without that headroom precision collapses as measured in E1′ (§6.7). Quantifying the cost-parity crossover point is deferred to future work with a DPDK-class traffic generator.

7.7. RAT manifest lifecycle

The RAT is not a hard-coded file. We sketch an operational pipeline: the operator's firmware deployment system (for example, Ansible, GitHub Actions, or a vendor portal API) emits a `rat.json` fragment per rollout that is merged into the active manifest and loaded into the controller on inotify hot-reload (or a 5-second polling fallback when inotify is unavailable). Two distinct RAT failure modes are quantified. First, integrity: under E12b's signed-manifest tamper test, a single-byte flip on the detached signature is rejected by the controller and last-known-good policy holds across 160/160 post-tamper events with 0 false positives across 3000 valid-signature events. The reload path is therefore fail-safe. With kernel inotify enabled, across 20 trials the controller logged a `RAT reload REJECTED` event in 20/20 trials and re-loaded the last-known-good manifest in `signed=True` mode in 20/20 trials; the controller never silently fell through to unsigned mode. The fail-safe behavior itself is independent of the trigger-observation rate. Second, freshness: a stale RAT (for example, a manifest 24 hours behind the actual rollout schedule) degrades precision because genuine rollouts fall outside the active time window, and the arbiter then converts their R1 fires to DROP overrides. On a benign-only workload this is a pure-loss failure mode: every R1 fire under a stale RAT is a false positive, so an operator running on a stale manifest is worse off than one running with the RAT disabled on the same benign workload. The lifecycle plumbing (signed manifests, inotify hot-reload with polling fallback, stale-window alerts) is therefore essential.

Lifecycle matrix (E22). On the deployed gated build we re-ran the active-authorized lifecycle state (signed RAT hot-reload, per-event reset, ten trials): all 80 authorized-rollout events PASS with zero false positives, reproducing E12. The remaining four states of the original matrix (pre-rollout, post-expiry, active-unauthorized off-target, and max-concurrent) probe enforcement the deployed data plane does not perform: an authorized source with a valid version, size, and target that deviates only in schedule or concurrency carries no unauthorized-source or oversize signal, trips no terminal rule, and is admitted rather than quarantined. This is the documented R5-gating scope (the defended space is unauthorized-source campaigns and unauthorized rollbacks, not off-parameter operation from an already-authorized source, §6.4), and schedule freshness is enforced at the manifest layer through E12b's signed-reload and stale-window path (§6.3), not by a data-plane rule. Two harness constraints make a full deployed-build matrix uninformative: per-event reset isolates single packets, so the R5 fleet-fanout counter cannot fire by construction, and although the controller enforces `max_concurrent_targets` (§4.3), the per-event reset keeps the concurrent-session count from accumulating, so the cap cannot be exercised in this harness. We therefore report only the in-scope active-authorized confirmation and do not treat the off-parameter states as detection events; a containment-oriented matrix would require the 5-tuple-indexed hold register of §7.2 behind a P4 recompile, left to future work.

Rollback decisions ride the same lifecycle. Gate B (§4.3.2) expresses an authorized downgrade campaign as an optional `rollback_window` field on the per-rollout manifest fragment, and inherits the manifest's signing and refresh pipeline. The defaults favor the operator: a manifest without `rollback_window` entries keeps R6 terminal, and a stale or absent manifest degrades to R6-as-terminal rather than silent suppression. Gate B

sits behind the same signed-manifest boundary that Gate A already relies on.

8. Conclusion

We described OTA-Shield, a hardware-measured reference architecture for in-network admission of authorized OTA rollouts and bounded detection of OTA firmware attacks against battery energy storage systems, implemented on an Intel Tofino 1 programmable switch. The design places an OTA parser and fleet-cardinality counter on the transit path where endpoint signature checks are blind, and a policy-aware control-plane arbiter (Gate A, Gate B) that consults a Rollout-Authorization Table on a cleartext segment after TLS or VPN termination.

On a 50-unit emulated BMS testbed the data-plane rules hold attack precision at 1.0 without control-plane help (E4 ablation on the gated build); the RAT arbiter's measured role is admitting operator-authorized rollbacks that the version-monotonicity rule (R6) would otherwise drop (E19). The held-out mimicry study reports each rule's measured blind zone (adaptive mimicry drops coverage to 1.7% of attack events), and the capacity sweep reports when the session-override table saturates under sustained install pressure. The design also detects signed rollback attempts through a per-BMS version-monotonicity rule (R6), with authorized downgrade exceptions handled explicitly through Gate B.

The contribution is the arbitration architecture: a data-plane rule set gated so that authorized rollouts never reach the fanout counter, and a RAT arbiter that admits the operator rollbacks the version rule would otherwise drop. A second contribution is the deployment-boundary characterization: the evaluation measures where the architecture holds and where brokered MQTT, TCP segmentation, parser variation, and adaptive mimicry require an extension. Two scopes bound the result. The $F_1 = 0.996$ headline covers the designed fleet-fanout family; an adversary who reshapes cadence faces the 1.7% mimicry coverage instead. Switch line-rate is not measured here, and the end-to-end F_1 is bound by a scapy generator ceiling of $\approx$2,200 pps. The reference-channel profile, the cadence-gated R1, and the bounded Suricata comparison are detailed in §7. Future work includes variable-length topic parsing, a set-associative session table for larger fleets, held-out OTA-channel variants (binary-diff firmware, broker-relayed MQTT, non-MQTT transports), and operational integration with complementary telemetry monitoring. A cross-protocol side-effect rule (R7) that correlates post-flash bursts in NTP, DNS, and DHCP within a short window is a further extension that would cover encrypted OTA channels whose payload we cannot parse.

Data availability

Code and data will be released at an anonymized repository upon acceptance; URL withheld for double-blind review.

Declaration of competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the writing process

The author used AI-assisted coding tools to draft code and AI for basic grammar correction. The author reviewed and edited all output and takes full responsibility for the publication.

A. E8 per-trial results

Table 17 reports the raw per-trial outcomes for the 20 E8 stochastic trials summarized in §6.3. Across 20 trials the detector produces $F_1 = 0.9962 \pm 0.0075$ (mean $\pm$ stdev; range $[0.9744, 1.0000]$), with dispersion concentrated in three trials that miss one to three of 60 attack events. Precision is exactly 1.0 on every trial. These numbers underlie the aggregated `0.996` [`0.993`, `0.999`] result reported in the body; readers who want to assess dispersion directly rather than rely on the bootstrap CI can do so from this table.

Table 17
E8 per-trial confusion-matrix counts and derived metrics. TP/FN/FP/TN are raw event counts; precision, recall, and F1 are derived per-trial.

Trial	TP	FN	FP	TN	Prec.	Rec.	F1
1	59	1	0	30	1.0000	0.9833	0.9916
2	60	0	0	30	1.0000	1.0000	1.0000
3	60	0	0	30	1.0000	1.0000	1.0000
4	60	0	0	30	1.0000	1.0000	1.0000
5	60	0	0	30	1.0000	1.0000	1.0000
6	60	0	0	30	1.0000	1.0000	1.0000
7	60	0	0	30	1.0000	1.0000	1.0000
8	60	0	0	30	1.0000	1.0000	1.0000
9	57	3	0	30	1.0000	0.9500	0.9744
10	58	2	0	30	1.0000	0.9667	0.9831
11	60	0	0	30	1.0000	1.0000	1.0000
12	58	2	0	30	1.0000	0.9667	0.9831
13	60	0	0	30	1.0000	1.0000	1.0000
14	59	1	0	30	1.0000	0.9833	0.9916
15	60	0	0	30	1.0000	1.0000	1.0000
16	60	0	0	30	1.0000	1.0000	1.0000
17	60	0	0	30	1.0000	1.0000	1.0000
18	60	0	0	30	1.0000	1.0000	1.0000
19	60	0	0	30	1.0000	1.0000	1.0000
20	60	0	0	30	1.0000	1.0000	1.0000
mean	59.55	0.45	0.0	30.0	1.0000	0.9925	0.9962
stdev	0.89	0.89	0.0	0.00	0.0000	0.0148	0.0075

B. E19′ first-contact recall taxonomy

Rollback detection (R6) requires a prior authenticated reference version for the targeted BMS: an attack packet to a device with no established baseline produces a no-decision (ND) rather than a HOLD, because R6 has no high-water mark to compare against. When a trial's Phase-1 baseline covers fewer devices than the attack set targets, the uncovered devices have no stored version, so attack packets to them generate a parse-level digest rather than a HOLD digest and the controller makes no decision. This is a fundamental property of a version-monotonicity detector, not a tuning artifact. We report two recall framings on the deployed gated build (460 rollback attacks across 20 trials; the detector drops 394 of them):

Framing (i), ND-excluded (lenient). Recall over attack events on devices with an established Phase-1 baseline: $394/406 = 0.970$ (Wilson 95% CI $[0.949, 0.983]$). This matches the detector's stated operating condition, a BMS whose baseline R6 has already recorded. The 12 reachable misses are rollbacks that reached the controller but were not dropped.

Framing (ii), strict (headline). Counting the 54 first-contact ND events as false negatives: $394/460 = 0.857$ (Wilson 95% CI $[0.822, 0.886]$). This is the figure we headline, treating any undetected attack packet as a miss regardless of first-contact status, per evaluation-integrity guidance for security detectors [41, 42].

The first-contact ND rate is 54/460. Precision is 1.000 with zero benign false positives across 340 legitimate events (§6.4). The un-gated build's apparent near-perfect recall was an artifact of the `hold_armed_reg` cascade incidentally dropping first-contact rollbacks; on the deployed gated build that cascade is inert (§6.4), so the cold-start misses surface as the gap between strict and lenient recall.

C. Build provenance and the R5-gating revision

The results in this paper were collected on two successive binaries, and this appendix records which experiment ran on which so that each number can be tied to a fixed configuration. The deployed build gates the fleet-fanout counter (R5) on the unauthorized-source flag (`r2_fired`) and arms the per-source `hold_armed_reg` gate only on R5; because R5 sits behind the terminal R2 drop, that gate is inert on reachable input (§6.4, §7.2). An earlier un-gated build armed the per-source gate on *any* rule fire. On that build a compressed benign rollout that tripped the per-BMS replay rule (R1) would arm the gate and drop the source's subsequent packets at line rate before the arbiter could clear them, producing benign-rollout false positives. Restricting the gate's arming to R5 removes this behavior, and E12, E19, and E19′ were re-measured on the deployed build, where they report zero benign-rollout false positives (§6.3, §6.4).

One consequence of the revision is the rollback recall the deployed build reports. The un-gated build recorded a near-perfect rollback recall, but that figure was inflated: the per-source gate incidentally dropped first-contact rollbacks that R6 itself cannot catch, because a first contact to a BMS has no stored version high-water mark. On the deployed build those cold-start cases surface as the gap between strict recall (0.857, counting first-contact events as misses) and lenient recall (0.970); we report the strict figure (§6.4, Appendix B).

The five-state lifecycle matrix (E22, §7.7) was originally completed only on the un-gated binary, where the per-source gate pre-empts the arbiter (152 of 160 benign rollout events dropped, a 95% false-positive rate). We re-ran the in-scope active-authorized state on the deployed gated build (0 false positives across 80 events, confirming the fix); the remaining states probe off-parameter authorized operation that the gated build does not quarantine (§7.7), and a containment matrix that also exercises a per-source hold role would require the 5-tuple-indexed hold register of §7.2 behind a P4 recompile. The integrity and freshness lifecycle failure modes are covered by E12b (§6.3). Table 18 summarizes the per-experiment configuration.

Table 18
Per-experiment build provenance. "Deployed gated" is the final binary (R5 r2-gated, per-source hold gate inert, 5-tuple-keyed override table); "un-gated" is the earlier binary used before the R5-gating revision. The last column marks whether the result is part of the deployed-build story or is auxiliary/motivating only.

Experiment(s)	Binary	R5 gating	Deployed story
E1, E2, E6, E8, E14	deployed gated	r2-gated	yes
E4 (arbiter ablation)	deployed gated	r2-gated	yes
E12, E12b	deployed gated	r2-gated	yes
E19, E19′	deployed gated	r2-gated	yes
E9, E21, E23, E20	deployed gated	r2-gated	yes
E10 (incl. symmetric)	deployed gated	r2-gated	yes
E11, E13, E18	deployed gated	r2-gated	yes
E7b (long-horizon)	deployed gated	r2-gated	yes
E1′, E8′ (200-BMS)	earlier binary	n/a	scope only (cap-limited)
E22 active-authorized	deployed gated	r2-gated	yes (confirms fix)
E22 full matrix	un-gated	ungated	motivating only

The override table is 5-tuple-keyed on the deployed build throughout (§6.7); E13 additionally characterizes the optional source-IP aggregation variant. E7b was re-run on the deployed gated build (§6.5): over the 8 h trace it records 0 benign false positives and catches all 100 rollback attacks, so it now reports on the deployed binary rather than an un-gated auxiliary run.

References

[1] Peter Fox-Penner, Phil Tonkin, Justin Pascale, Noah Rauschkolb, and Purvaansh Lohiya. Securing battery energy

storage systems from cyberthreats: Best practices and trends for protecting critical energy infrastructure. Technical report, The Brattle Group and Dragos, Inc., December 2025. URL https://www.brattle.com/insights-events/publications/securing-battery-energy-storage-systems-from-cyberthreats-best-practices-and-trends-for-protecting-critical-energy-infrastructure/. Supported by Fluence Energy.

[2] Frans Öhrström, Joakim Oscarsson, Zeeshan Afzal, János Dani, and Mikael Asplund. From balance to breach: cyber threats to battery energy storage systems. *Energy Informatics*, 8(1), March 2025. ISSN 2520-8942. doi: 10.1186/s42162-025-00499-4. URL http://dx.doi.org/10.1186/s42162-025-00499-4.

[3] Megan Jordan Culler. Threat landscape for BESS and IBR. In *IEEE PES Grid Edge Technologies Conference & Exposition*, January 2025. Idaho National Laboratory report INL/CON-25-82636-Rev. 0, contract DE-AC07-05ID14517.

[4] Brendan Moran, Hannes Tschofenig, Henk Birkholz, and Koen Zandberg. RFC 9019: A firmware update architecture for Internet of Things. IETF Informational RFC, April 2021. URL https://www.rfc-editor.org/rfc/rfc9019.

[5] Robert Lorch, Daniel Larraz, Cesare Tinelli, and Omar Chowdhury. A comprehensive, automated security analysis of the Uptane automotive over-the-air update framework. In *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses (RAID '24)*, pages 594–612. ACM, September 2024. doi: 10.1145/3678890.3678927. URL http://dx.doi.org/10.1145/3678890.3678927.

[6] Sandia National Laboratories and SunSpec Alliance. IEEE 2030.5 implementation guide for distributed energy resources: Certificate, authentication, and revocation handling. Technical report, Sandia National Laboratories, 2024. URL https://sunspec.org/2030-5-csip/. Implementation guidance leaves OCSP/CRL revocation handling to the deploying aggregator; no online revocation profile is mandated for the OTA path.

[7] Qinwen Hu, Se-Young Yu, and Muhammad Rizwan Asghar. Analysing performance issues of open-source intrusion detection systems in high-speed networks. *Journal of Information Security and Applications*, 51:102426, 2020. doi: 10.1016/j.jisa.2019.102426. URL https://doi.org/10.1016/j.jisa.2019.102426. Quantifies Snort/Suricata drop behavior at 1-40 Gbps; AF_Packet reaches 60 Gbps before collapse.

[8] Abdul Waleed, Abdul Fareed Jamali, and Ammar Masood. Which open-source IDS? Snort, Suricata or Zeek. *Computer Networks*, 213:109116, August 2022. doi: 10.1016/j.comnet.2022.109116. URL https://doi.org/10.1016/j.comnet.2022.109116. Benchmarks resource use, throughput, packet drop, and detection across Snort, Suricata, Zeek.

[9] Rodrigo D. Trevizan. Cybersecurity of battery energy storage systems. Technical Report OSTI-1855330, Sandia National Laboratories, 2022. URL https://www.osti.gov/servlets/purl/1855330.

[10] Md Zahidul Islam, Yuzhang Lin, Vinod M. Vokkarane, and Venkatesh Venkataramanan. Cyber-physical cascading failure and resilience of power grid: A comprehensive review. *Frontiers in Energy Research*, 11, February 2023. ISSN 2296-598X. doi: 10.3389/fenrg.2023.1095303. URL http://dx.doi.org/10.3389/fenrg.2023.1095303.

[11] Frederik Hauser, Marco Häberle, Daniel Merling, Steffen Lindner, Vladimir Gurevich, Florian Zeiger, Reinhard Frank, and Michael Menth. A survey on data plane programming with P4: Fundamentals, advances, and applied research. *Journal of Network and Computer Applications*, 212:103561, 2023. doi: 10.1016/j.jnca.2022.103561. URL https://arxiv.org/abs/2101.10632. Preprint arXiv:2101.10632.

[12] Zaoxing Liu, Hun Namkung, Georgios Nikolaidis, Jeongkeun Lee, Changhoon Kim, Xin Jin, Vladimir Braverman, Minlan Yu, and Vyas Sekar. Jaqen: A high-performance switch-native approach for detecting and mitigating volumetric DDoS attacks with programmable switches. In *Proceedings of the 30th USENIX Security Symposium (USENIX Security '21)*, pages 3829–3846. USENIX Association, August 2021. ISBN 978-1-939133-24-3. URL https://www.usenix.org/conference/usenixsecurity21/presentation/liu-zaoxing. Switch-native DDoS defense demonstrated at 380 Gbps line rate on Tofino.

[13] Menghao Zhang, Guanyu Li, Shicheng Wang, Chang Liu, Ang Chen, Hongxin Hu, Guofei Gu, Qi Li, Mingwei Xu, and Jianping Wu. Poseidon: Mitigating volumetric DDoS attacks with programmable switches. In *Proceedings of the 27th Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, February 2020. Internet Society. doi: 10.14722/ndss.2020.24007. URL https://www.ndss-symposium.org/ndss-paper/poseidon-mitigating-volumetric-ddos-attacks-with-programmable-switches/. Tofino-based DDoS mitigation; includes line-rate stress evaluation methodology.

[14] Reuben Samson Raj and Dong Jin. Dynamic data driven security framework for industrial control networks using programmable switches. In *Dynamic Data Driven Applications Systems (DDDAS 2024)*, pages 153–161. Springer Nature Switzerland, August 2025. ISBN 9783031948954. doi: 10.1007/978-3-031-94895-4_16. URL http://dx.doi.org/10.1007/978-3-031-94895-4_16.

[15] Yaying Chen, Siamak Layeghy, Liam Daly Manocchio, and Marius Portmann. P4-NIDS: High-performance network monitoring and intrusion detection in P4, 2024. URL https://arxiv.org/abs/2411.17987.

[16] Bui Ngoc Thanh Binh, Pham Hoai Luan, Le Vu Trung Duong, Vu Tuan Hai, and Yasuhiko Nakashima. A protocol-aware P4 pipeline for MQTT security and anomaly mitigation in edge IoT systems, 2026. URL https://arxiv.org/abs/2501.07536. Accepted at ICOIN 2026.

[17] Abdulmohsen Almalawi, Adil Fahad, Zahir Tari, Asif Irshad Khan, Nouf Alzahrani, Sheikh Tahir Bakhsh, Madini O. Alassafi, Abdulrahman Alshdadi, and Sana Qaiyum. Add-on anomaly threshold technique for improving unsupervised intrusion detection on SCADA data. *Electronics*, 9(6): 1017, June 2020. doi: 10.3390/electronics9061017. URL https://doi.org/10.3390/electronics9061017. Data-driven empirical threshold derivation for SCADA anomaly scores; mitigates parameter sensitivity.

[18] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, Funchal, Portugal, January 2018. SciTePress. doi: 10.5220/0006639801080116. URL https://doi.org/10.5220/0006639801080116. CIC-IDS2017 dataset and evaluation framework; widely used baseline for IDS threshold and boundary studies.

[19] Nuvation Energy. *Communication Protocol Reference Manual (Revision 2.2)*. Nuvation Energy, May 2020. URL https://nuvationenergy.com/wp-content/uploads/2022/05/Nuvation-Energy-Communication-Protocol-Reference_2.2.pdf. Document ID: NE-RM-005.

[20] MESA Standards Alliance and SunSpec Alliance. MESA-Device/SunSpec energy storage model (draft 3). Industry specification, 2016. URL https://mesastandards.org/mesa-standards/.

[21] North American Electric Reliability Corporation. *NERC CIP Critical Infrastructure Protection Standards*. NERC, 2023. URL https://www.nerc.com/pa/Stand/Pages/CIPStandards.aspx. Applicability depends on Bulk Electric System role; partial applicability for generation <20 MW.

[22] National Fire Protection Association. *NFPA 855: Standard for the Installation of Stationary Energy Storage Systems*. NFPA, 2023. URL https://www.nfpa.org/product/nfpa-855-standard/p0855code. Includes cybersecurity section referencing UL 2900-1.

[23] International Electrotechnical Commission. *IEC 62443: Industrial Communication Networks – IT Security for Networks and Systems*. IEC/ISA, 2018. URL https://www.isa.org/standards-and-publications/isa-standards/isa-iec-62443-series-of-standards. Security Levels SL1-SL4; Foundational Requirements FR1-FR7.

[24] Forescout Vedere Labs. SUN:DOWN – destabilizing the grid via orchestrated exploitation of solar power systems. Technical report, Forescout Technologies, March 2025. URL https://www.forescout.com/blog/grid-security-new-vulnerabilities-in-solar-power-systems-exposed/. 46 new vulnerabilities in Sungrow, Growatt, SMA Solar; 32% CVSS 9.8-10.0.

[25] Lumen Black Lotus Labs. Derailing the Raptor Train. Technical report, Lumen Technologies, September 2024. URL https://blog.lumen.com/derailing-the-raptor-train/. 260K+ compromised SOHO/IoT devices attributed to Integrity Technology Group (PRC).

[26] Jiahao Wu, Heng Pan, Penglai Cui, Yiwen Huang, Jianer Zhou, Peng He, Yanbiao Li, Zhenyu Li, and Gaogang Xie. Patronum: In-network volumetric DDoS detection and mitigation with programmable switches. In *Computer Security – ESORICS 2024*, pages 187–207. Springer Nature Switzerland, 2024. ISBN 9783031709036. doi: 10.1007/978-3-031-70903-6_10. URL http://dx.doi.org/10.1007/978-3-031-70903-6_10.

[27] Ali AlSabeh, Elie Kfoury, Jorge Crichigno, and Elias Bou-Harb. P4DDPI: Securing P4-programmable data plane networks via DNS deep packet inspection. In *Proceedings 2022 Workshop on Measurements, Attacks, and Defenses for the Web (MADWeb '22)*. Internet Society, 2022. doi: 10.14722/madweb.2022.23012. URL http://dx.doi.org/10.14722/madweb.2022.23012.

[28] Albert Gran Alcoz, Martin Strohmeier, Vincent Lenders, and Laurent Vanbever. Aggregate-based congestion control for pulse-wave DDoS defense. In *Proceedings of the ACM SIGCOMM 2022 Conference*, pages 693–706, Amsterdam, Netherlands, August 2022. ACM. doi: 10.1145/3544216.3544263. URL https://doi.org/10.1145/3544216.3544263. ACC-Turbo: line-rate pulse-wave DDoS mitigation on Tofino; stress-test methodology vs pulse-wave attackers.

[29] Qiao Kang, Jiarong Xing, and Ang Chen. Programmable in-network security for context-aware BYOD policies. In *Proceedings of the 29th USENIX Security Symposium (USENIX Security '20)*, pages 2073–2090. USENIX Association, August 2020. ISBN 978-1-939133-17-5. URL https://www.usenix.org/conference/usenixsecurity20/presentation/kang. Poise: context-aware BYOD policy enforcement on programmable switches without control-plane round-trips; Tofino SDE 8.4.

[30] Jiarong Xing, Qiao Kang, and Ang Chen. NetWarden: Mitigating network covert channels while preserving performance. In *Proceedings of the 29th USENIX Security Symposium (USENIX Security '20)*, pages 2055–2072. USENIX Association, August 2020. ISBN 978-1-939133-17-5. URL https://www.usenix.org/conference/usenixsecurity20/presentation/xing. NetWarden: in-network covert-channel mitigation at line rate; fastpath/slowpath on programmable switch; 3–101 ns overhead vs baseline.

[31] Sankalp Mittal. CyberSentinel: Efficient anomaly detection in programmable switch using knowledge distillation, 2024. URL https://arxiv.org/abs/2412.16693.

[32] Luca Verderame, Antonio Ruggia, and Alessio Merlo. PARIOT: Anti-repackaging for IoT firmware integrity, 2023. URL https://arxiv.org/abs/2109.04337. Published in Journal of Network and Computer Applications 2023.

[33] Pegah Nikbakht Bideh and Christian Gehrmann. RoSym: Robust symmetric key based IoT software upgrade over-the-air. In *Proceedings of the 4th Workshop on CPS & IoT Security and Privacy (CPSIoTSec '22)*, pages 35–46. ACM, November 2022. doi: 10.1145/3560826.3563381. URL http://dx.doi.org/10.1145/3560826.3563381.

[34] Insu Oh, Mahdi Sahlabadi, Kangbin Yim, and Sunyoung Lee. Threat classification and vulnerability analysis on 5G firmware over-the-air updates for mobile and automotive platforms. *Electronics*, 14(10):2034, May 2025. ISSN 2079-9292. doi: 10.3390/electronics14102034. URL http://dx.doi.org/10.3390/electronics14102034.

[35] Muhammad Ali, Yasir Saleem, Sadaf Hina, and Ghalib A. Shah. DDoSViT: IoT DDoS attack detection for fortifying firmware over-the-air (OTA) updates using vision transformer. *Internet of Things*, 30:101527, March 2025. ISSN 2542-6605. doi: 10.1016/j.iot.2025.101527. URL http://dx.doi.org/10.1016/j.iot.2025.101527.

[36] Liang Cai, Yulong Chen, Weihong Cai, Yong Chen, and Jiaming Liu. A combination of CUSUM-EWMA for anomaly detection in time series data. In *2015 10th International Conference on Computer Science & Education (ICCSE)*, pages 451–454. IEEE, 2015. doi: 10.1109/ICCSE.2015.7250287. URL https://doi.org/10.1109/ICCSE.2015.7250287. Hybrid CUSUM-EWMA control chart for streaming anomaly detection; baseline for empirical threshold schemes.

[37] Qin Lin, Sridhar Adepu, Sicco Verwer, and Aditya Mathur. TABOR: A graphical model-based approach for anomaly detection in industrial control systems. In *Proceedings of the 2018 ACM Asia Conference on Computer and Communications Security (ASIACCS)*, pages 525–536. ACM, 2018. doi: 10.1145/3196494.3196546.

[38] Jesse Davis and Mark Goadrich. The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 233–240. ACM, 2006. doi: 10.1145/1143844.1143874.

[39] Dominik Kus, Eric Wagner, Jan Pennekamp, Konrad Wolsing, Ina Berenice Fink, Markus Dahlmanns, Klaus Wehrle, and Martin Henze. A false sense of security? revisiting the state of machine learning-based industrial intrusion detection. In *Proceedings of the 8th ACM Cyber-Physical System Security Workshop (CPSS)*, pages 73–84. ACM, 2022. doi: 10.1145/3494107.3522773.

[40] Marco Caselli, Emmanuele Zambon, and Frank Kargl. Sequence-aware intrusion detection in industrial control systems. In *Proceedings of the 1st ACM Workshop on Cyber-Physical System Security (CPSS)*, pages 13–24. ACM, 2015. doi: 10.1145/2732198.2732200.

[41] Feargus Pendlebury, Fabio Pierazzi, Roberto Jordaney, Johannes Kinder, and Lorenzo Cavallaro. TESSERACT: Eliminating experimental bias in malware classification across space and time. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 729–746. USENIX Association, 2019. URL https://www.usenix.org/conference/usenixsecurity19/presentation/pendlebury.

[42] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. Dos and don'ts of machine

learning in computer security. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3971–3988. USENIX Association, 2022. URL https://www.usenix.org/conference/usenixsecurity22/presentation/arp.

[43] C. J. van Rijsbergen. *Information Retrieval*. Butterworth-Heinemann, Newton, MA, USA, 2nd edition, 1979. Chapter 7 defines the parameterized F_β effectiveness measure.

[44] Steffen Lindner, Marco Häberle, and Michael Menth. P4TG: 1 Tb/s traffic generation for Ethernet/IP networks. *IEEE Access*, 11:17525–17535, February 2023. doi: 10.1109/ACCESS.2023.3246480. URL https://doi.org/10.1109/ACCESS.2023.3246480. Tofino-based line-rate traffic generator (10x100 Gb/s); measures loss, reordering, IAT, RTT for stress testing.

[45] Fabian Ihle, Etienne Zink, Steffen Lindner, and Michael Menth. Enhancements to P4TG: Protocols, performance, and automation, 2025. URL https://arxiv.org/abs/2501.17127. Extends P4TG to 10x400 Gb/s on Tofino 2; RFC 2544 automation; published at KuVS NetSoft 2025.